



서울시립대학교

Lec 5 - PWM

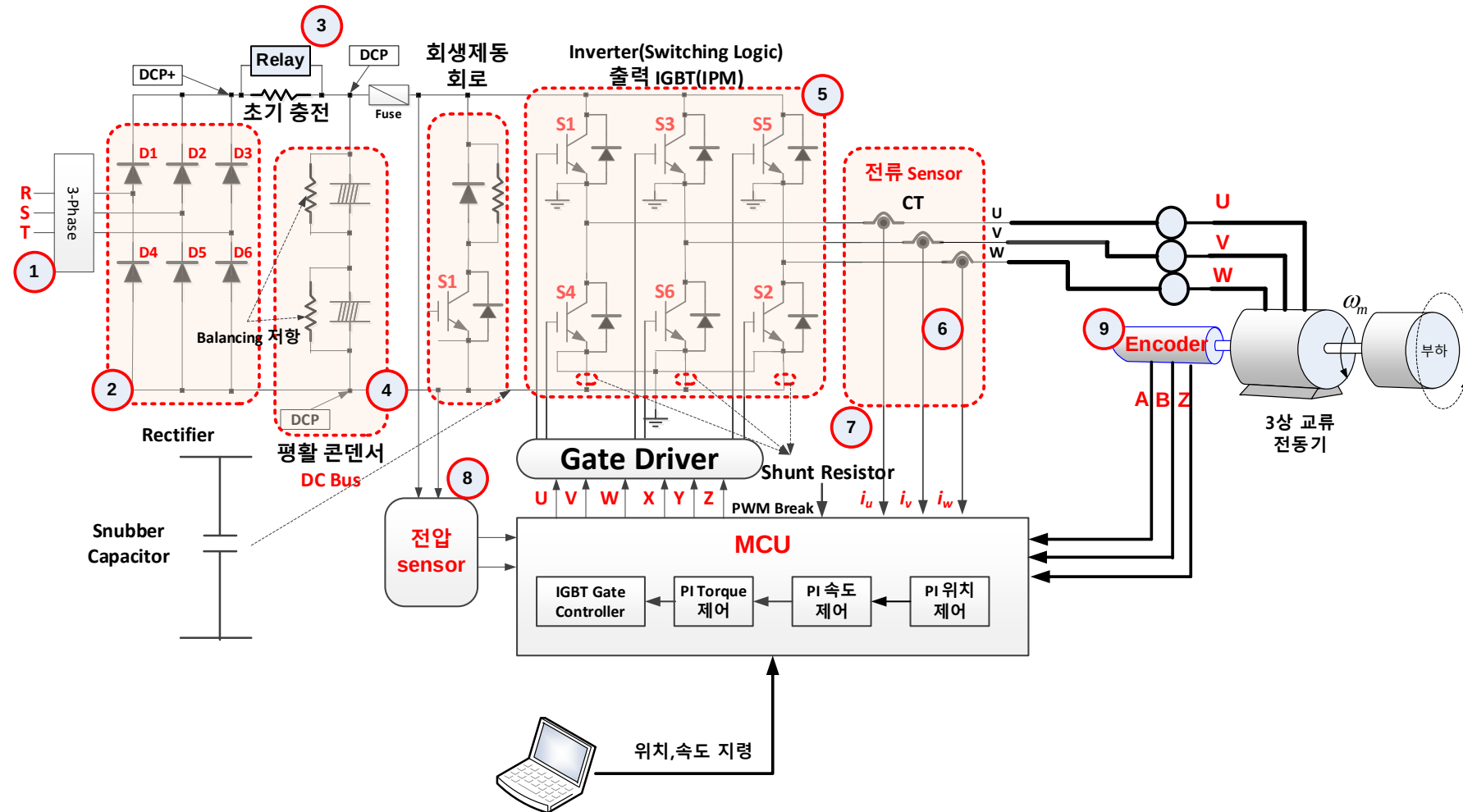
전력전자연구실 교수 이 승 환



PWM 필요성 - 1

✓ MCU에서 대전력을 사용하는 전동기를 구동하기 위한 스위치를 PWM을 발생하여 구동한다.

- 회전하는 모터에 대한 정보를 수집하여 원하는 목표치를 달성하도록 제어한다.
- Feedback control



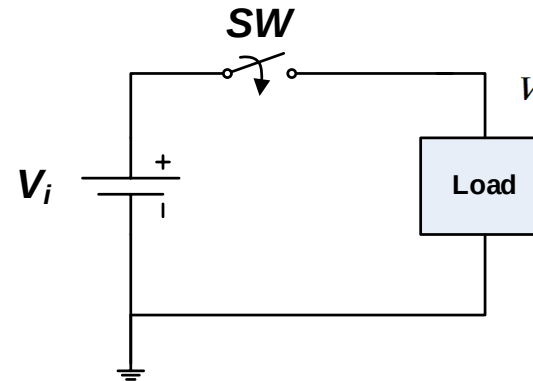
✓ (a)번과 같이 1 주기 동안의 PWM On/Off 비율을 구성하여 평균 전압을 제어한다.

✓ (b)번과 같이 회로를 구성하면,

- Switch S1, S2에 대한 On/Off를 이용하여 GND 기준의 minus 전압 즉, 교류를 생성할 수 있다.
- 삼각 반송파 즉, Carrier Wave 전압 V_c 와 지령 전압 V_{ref} 를 비교하여 1 주기에 대한 Switch S1, S2의 On/Off 시간을 설정한다.

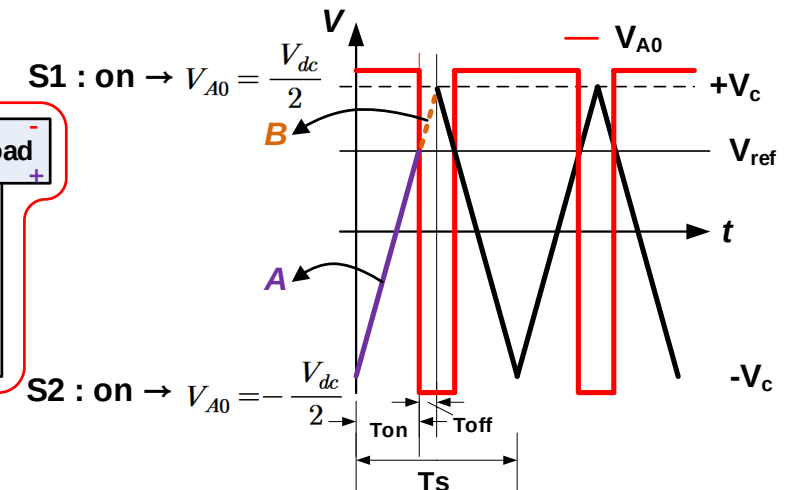
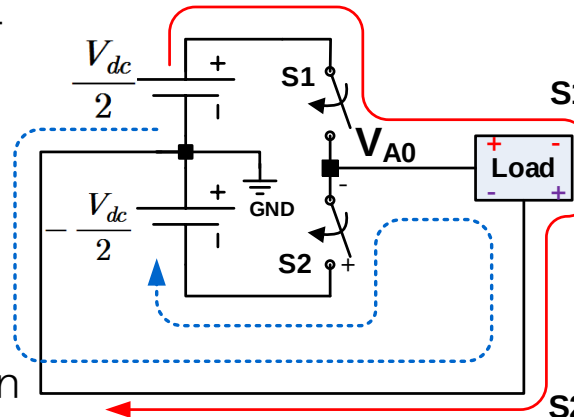
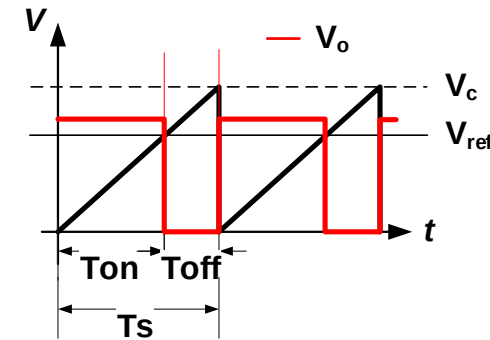
✓ 반 주기 동안 :

- 보라색 선(A) : $V_{ref} > V_c$, S1 switch On
- 주황색 선(B) : $V_{ref} < V_c$, S2 switch On



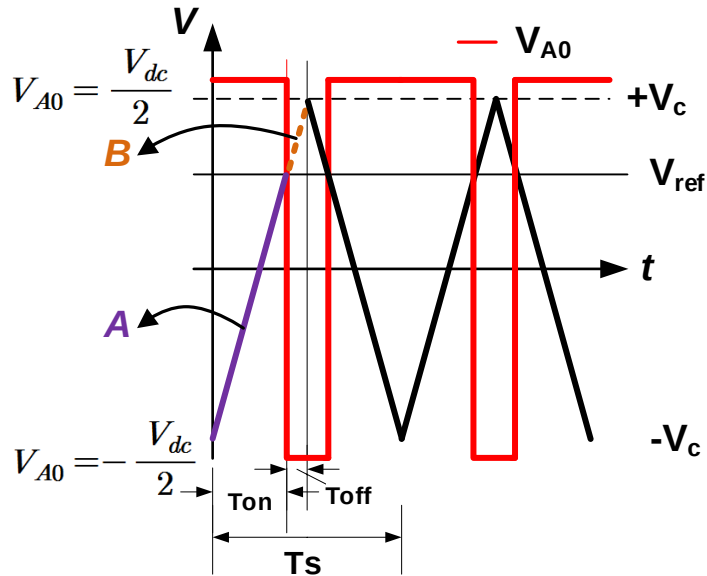
$$V_o = \frac{T_{on}}{T_s} V_i = \frac{V_{ref}}{V_c} V_i$$

(a) DC to DC



(b) DC to AC

✓ Sampling 1 주기 T_s 에 대한 평균 전압 :



Sampling 주기 T_s 에 대한 평균 전압 :

$$V_o = \frac{(T_{on} \times \frac{V_{dc}}{2}) \times 2 + (T_{off} \times -\frac{V_{dc}}{2}) \times 2}{T_s}$$

$$= \frac{T_{on} - T_{off}}{T_s} \times V_{dc}$$

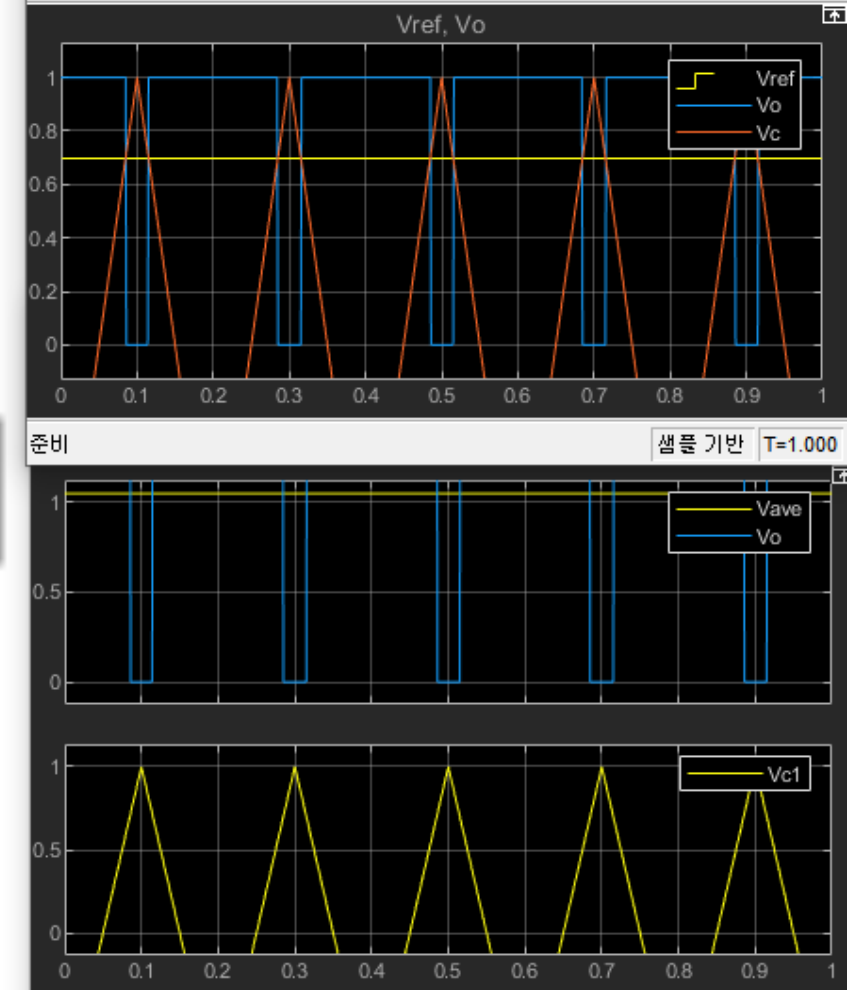
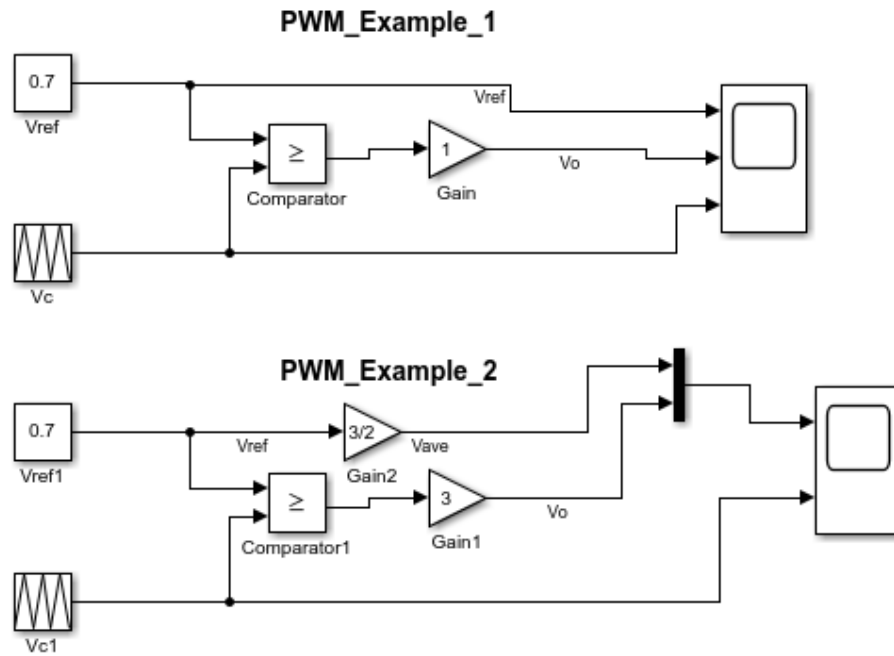
그리고, **A**라고 표시한 선에 대한 기울기는 $\frac{T_c}{V_c + V_{ref}}$ 이고, **B**라고 표시한 선에 대한 기울기는 $\frac{V_c - V_{ref}}{T_{off}}$ 이다. 이 2개의 선들은 동일한 기울기를 가지므로 다음과 같은 수식을 얻을 수 있다.

$$\frac{V_c + V_{ref}}{T_{on}} = \frac{V_c - V_{ref}}{T_{off}} \quad (\text{수식 67})$$

(수식 65)로부터 (수식 67)을 정리하면, 다음과 같이 (수식 68)을 얻을 수 있다.

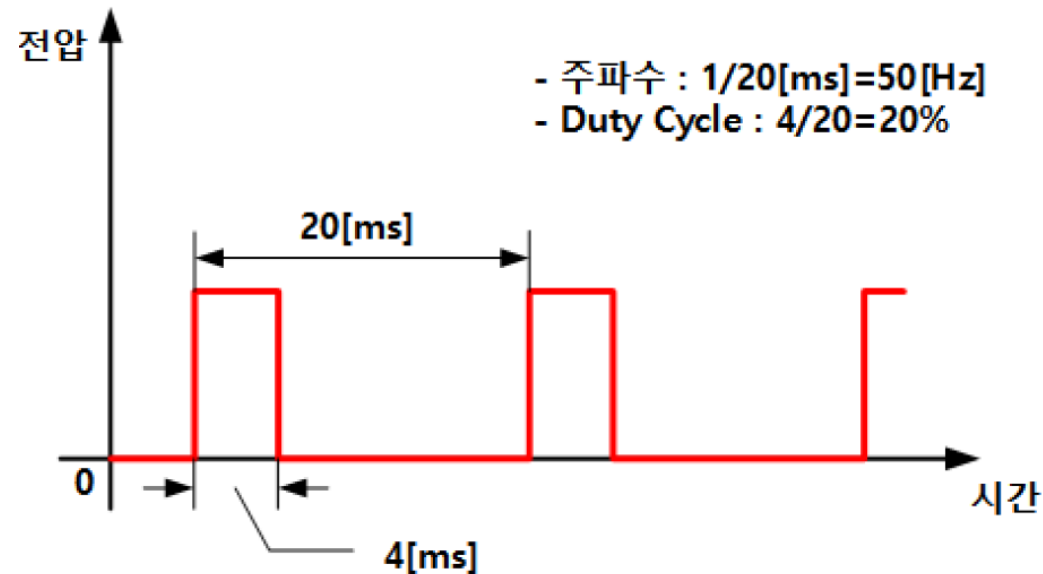
$$V_o = \frac{V_{ref}}{V_c} \times \frac{V_{dc}}{2} \quad (\text{수식 68})$$

✓ Sampling 1 주기 T_s 에 대한 평균 전압 :



✓ PWM 정리 :

- [그림 7.1-4]에서 보여준 PWM 파형의 특징(사양, specification).
 - 주기 : 20[ms](즉, 50[Hz], 주파수).
 - Duty Cycle : 1주기 전체 20[ms]에서 high level 구간은 4[ms] 즉, $4/20=1/5$ 로서 duty cycle은 20%.
 - Pulse Width Modulation 즉, High level 구간에 의미를 부여한다. 즉, duty cycle이 전달하려는 데이터이다.
- PWM 파형을 출력하기 위해서는 출력 port 필요!

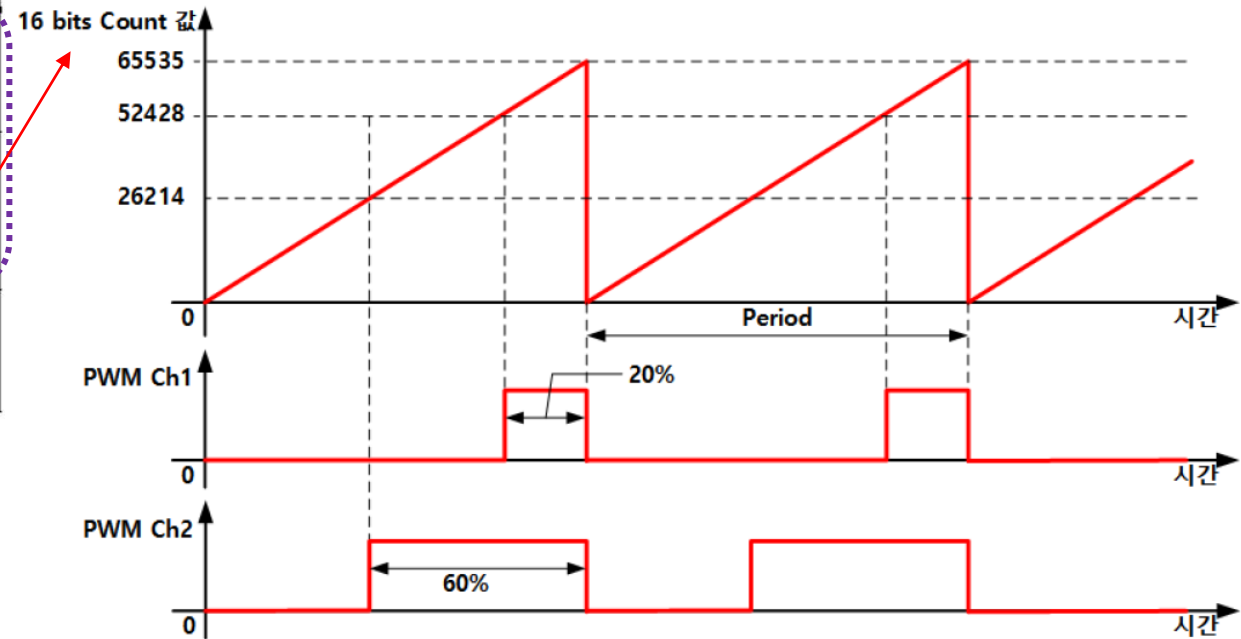


[그림 7.1-4] 전형적인 PWM 신호.

✓ PWM 정리 :

- Chapter 6에서 3 종류 Timer들 학습 :
 - General Purpose Timer와 Advanced Control Timer PWM 출력 port 제공.
 - Basic Timer는 출력 port가 없으므로 PWM 파형 출력 불가!

Timer	Counter resolution	Counter type	Prescaler factor	DMA request generation	Capture/compare channels	Complementary outputs
TIM1, TIM8	16-bit	Up, down, up/down	Any integer between 1 and 65536	Yes	4	Yes
TIM2, TIM3, TIM4, TIM5	16-bit	Up, down, up/down	Any integer between 1 and 65536	Yes	4	No
TIM6, TIM7	16-bit	Up	Any integer between 1 and 65536	Yes	0	No



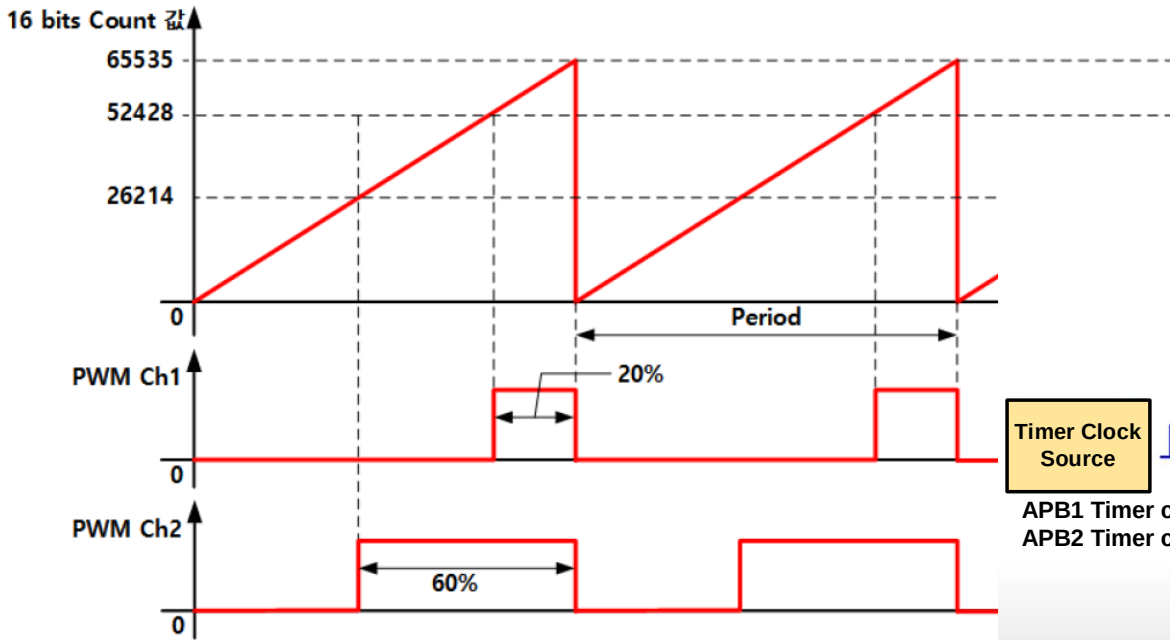
[그림 7.1-5] PWM 신호 생성(1).

✓ PWM 정리 :

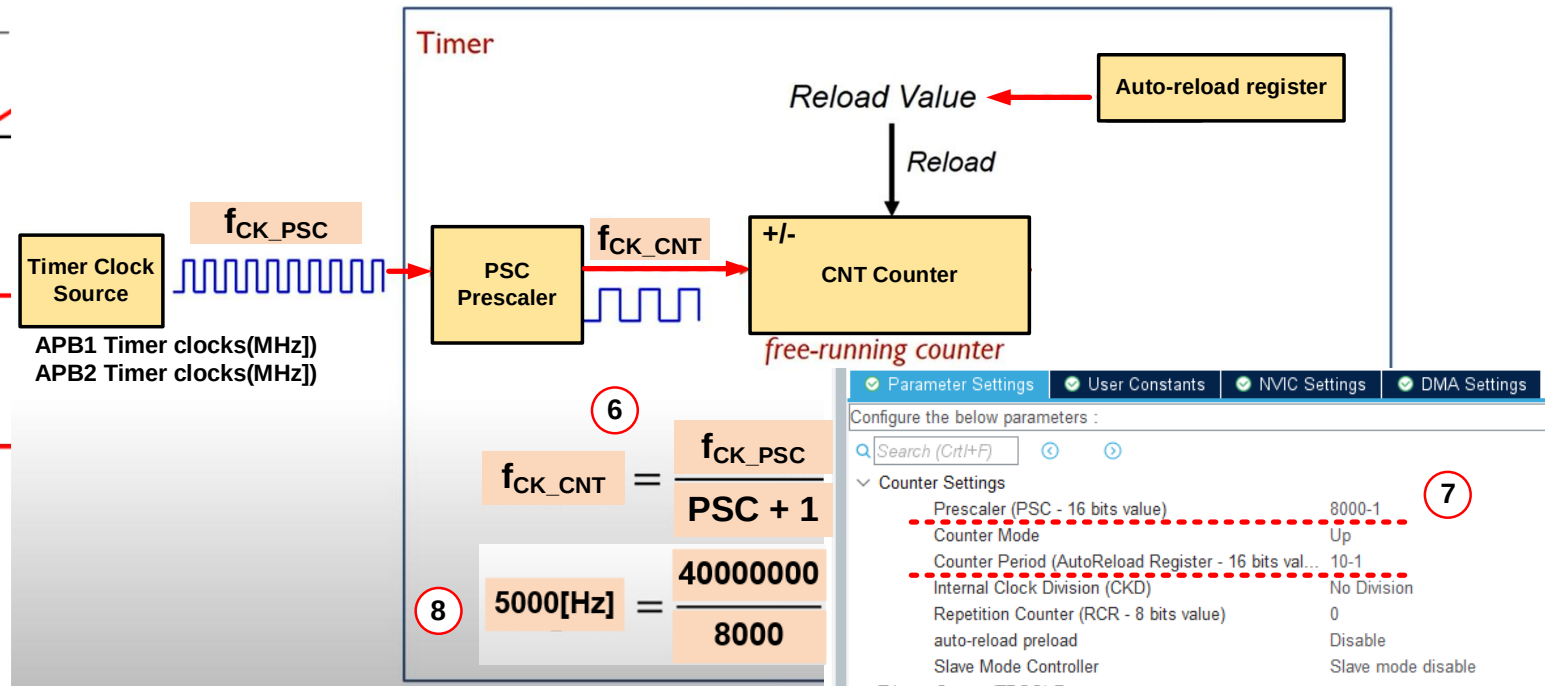
- 16bits Counter는 Timer의 CNT Counter Register에 저장되는 값 : 0~65535

✓ PWM Ch1 : 20% duty cycle

✓ PWM Ch2 : 60% duty cycle



[그림 7.1-5] PWM 신호 생성(1).



✓ PWM mode :

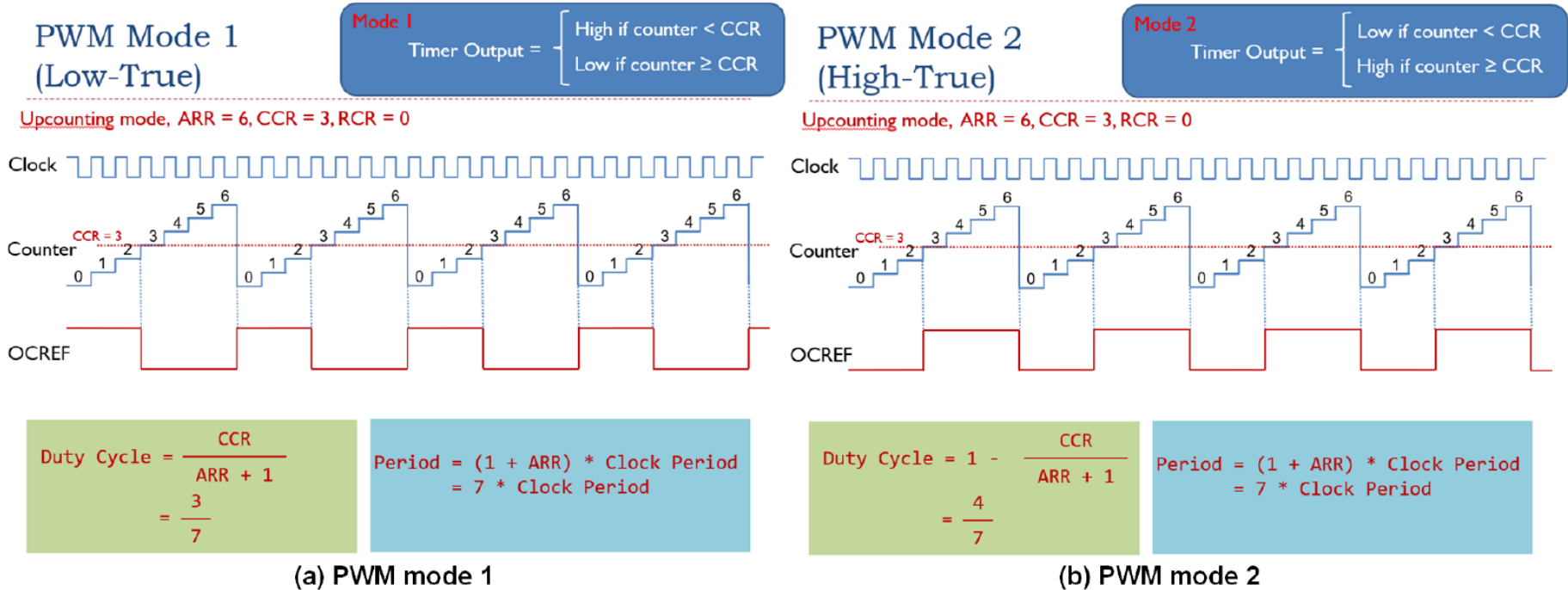
- CCR(Capture Compare Register)과 CNT register을 이용한 PWM 생성 mode.

① PWM Mode 1 :

CNT counter 값 < CCR 일 때는 High level 출력, 그렇지 않을 때는 Low level 출력.

② PWM Mode 2 :

CNT counter 값 < CCR 일 때는 Low level 출력, 그렇지 않을 때는 High level 출력.



간단한 PWM 생성 방법 및 실습 - 5

✓ 실습 :

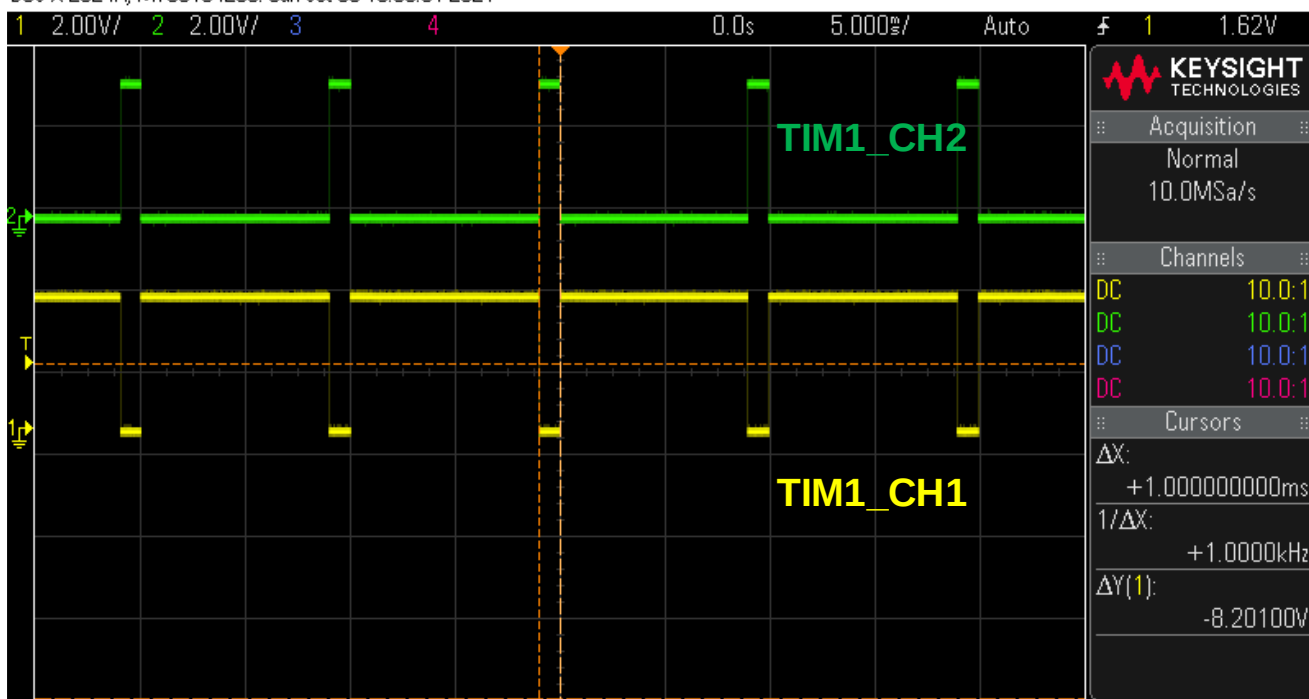
- TIM1
- Internal Clock
- PSC=8000-1
- ARR=100-1
- PWM Ch1 :
 - PWM Mode-1
 - Pulse : 90
- PWM Ch2 :
 - PWM Mode-2
 - Pulse : 90

The screenshot displays the STM32CubeMX configuration for TIM1. The left sidebar shows the 'Timers' category with TIM1 selected (marked with a red circle 1). The main configuration area shows the 'TIM1 Mode and Configuration' window. The 'Mode' tab is active, showing 'Clock Source' set to 'Internal Clock' (marked with a red circle 2). The 'Configuration' tab shows 'Parameter Settings' with 'Prescaler (PSC - 16 bits value)' set to 8000-1, 'Counter Mode' set to 'Up' (marked with a red circle 4), and 'Counter Period (AutoReload Register - 16 bits value)' set to 100-1. The 'PWM Generation Channel 1' settings are shown with 'Mode' set to 'PWM mode 1' and 'Pulse (16 bits value)' set to 90 (marked with a red circle 5). The 'PWM Generation Channel 2' settings are shown with 'Mode' set to 'PWM mode 2' and 'Pulse (16 bits value)' set to 90 (marked with a red circle 6). The right pinout diagram shows the PA9 and PA8 pins assigned to TIM1_CH2 and TIM1_CH1 respectively (marked with a red circle 3).

✓ 과제 : 아래 파형 오실로스코프로 확인 (조교 확인 필수)

- PWM TIM1_CH1 출력은 전체 10[ms]에서 1[ms]만 low level. duty cycle은 90%.
- PWM TIM1_CH2 출력은 전체 10[ms]에서 1[ms]만 high level. duty cycle은 10%.
- Timer와 달리 HAL_TIM_PWM_Start() 함수로 해당 timer를 enable, channel도 할당.

DSO-X 2024A, MY58104289: Sun Oct 03 16:50:01 2021



Main.c

```
#include "main.h"
#include "htim1.h"
#include "huart2.h"
#include "SystemClock_Config.h"
#include "MX_GPIO_Init.h"
#include "MX_USART2_UART_Init.h"
#include "MX_TIM1_Init.h"
int main(void)
{
    SystemClock_Config(&htim1);
    MX_TIM1_Init(&htim1);
    MX_USART2_UART_Init(&huart2);
    MX_GPIO_Init(&htim1);
    Error_Handler();
    #ifndef USE_FULL_ASSERT
    assert_failed(0, __FILE__, __LINE__);
    #endif
}
```

```
00071: /* USER CODE BEGIN 1 */
00072:
00073: /* USER CODE END 1 */
00074:
00075: /* MCU Configuration----- */
00076:
00077: /* Reset of all peripherals, Initializes the Flash interface */
00078: HAL_Init();
00079:
00080: /* USER CODE BEGIN Init */
00081:
00082: /* USER CODE END Init */
00083:
00084: /* Configure the system clock */
00085: SystemClock_Config();
00086:
00087: /* USER CODE BEGIN Sysinit */
00088:
00089: /* USER CODE END Sysinit */
00090:
00091: /* Initialize all configured peripherals */
00092: MX_GPIO_Init();
00093: MX_USART2_UART_Init();
00094: MX_TIM1_Init();
00095: /* USER CODE BEGIN 2 */
00096: HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
00097: HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
00098: /* USER CODE END 2 */
00099:
00100: /* Infinite loop */
00101: /* USER CODE BEGIN WHILE */
00102: while (1)
00103: {
00104:     /* USER CODE END WHILE */
```

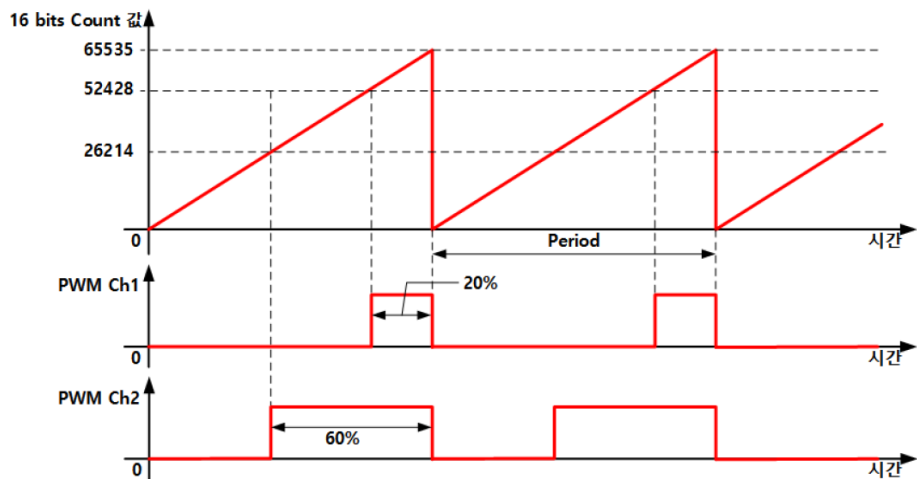
7

✓ PWM mode 2에서 CCR 값을 계산하는 방법.

다. 다음의 수식은 PWM mode 2에서 CCR 값을 계산하는 과정을 설명한 것이다. 여기서, MAX는 16bits ARR 값을 의미하고, DC는 Duty Cycle을 의미한다. 즉, high level 구간을 설정하기 위한 CCR 값은 다음과 같이 계산하면 된다.

$$DC = \frac{MAX - CCR}{MAX} \quad CCR = MAX - MAX \times DC = MAX \times (1 - DC) \quad (\text{식 7.1-2})$$

✓ 예를 들어서, PWM Ch2는 60%이다. 이때에 CCR2 register 저장할 값은 (식 7.1-2)로부터 다음과 같이 계산된다. (ARR = 65536-1로 설정한 경우)



$$CCR2 = MAX \times (1 - DC) = 65535 \times (1 - 0.6) = 26214 \quad (\text{식 7.1-3})$$

✓ timer 2의 CCR2 register에 26214를 설정해 주면, 60% duty cycle을 갖는 PWM 신호가 생성된다.

✓ PWM과 Timer 연동 방법 :

- Timer 2는 500[ms] 마다 interrupt 발생.
- Timer 1의 CNT(ARR) = 100인 경우.

Main.c

```
#include "main.h"
htim1
htim2
huart2
DutyR
SystemClock_Co
MX_GPIO_Init
MX_USART2_UART
MX_TIM1_Init
MX_TIM2_Init
main
SystemClock_Co
MX_TIM1_Init
MX_TIM2_Init
MX_USART2_UART
MX_GPIO_Init
HAL_TIM_PeriodE
Error_Handler
#ifdef USE_FULL_
assert_failed
#endif
```

```
00372: GPIO_InitStruct.Pull = GPIO_NOPULL;
00373: GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_MEDIUM;
00374: HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
00375:
00376: } ? end MX_GPIO_Init ?
00377:
00378: /* USER CODE BEGIN 4 */
00379: void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
00380: {
00381:     if(htim->Instance==TIM2) { // UI is happened every 50[ms]
00382:         HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_8);
00383:         TIM1->CCR1=DutyR;
00384:         TIM1->CCR2=DutyR;
00385:         if (DutyR%100) {
00386:             DutyR++;
00387:         } else {
00388:             DutyR=1;
00389:         }
00390:     }
00391: }
```

5

Main.c

```
#include "main.h"
htim1
htim2
huart2
DutyR
SystemClock_Co
MX_GPIO_Init
MX_USART2_UART
MX_TIM1_Init
MX_TIM2_Init
main
SystemClock_Co
MX_TIM1_Init
MX_TIM2_Init
MX_USART2_UART
MX_GPIO_Init
HAL_TIM_PeriodE
Error_Handler
#ifdef USE_FULL_
assert_failed
#endif
```

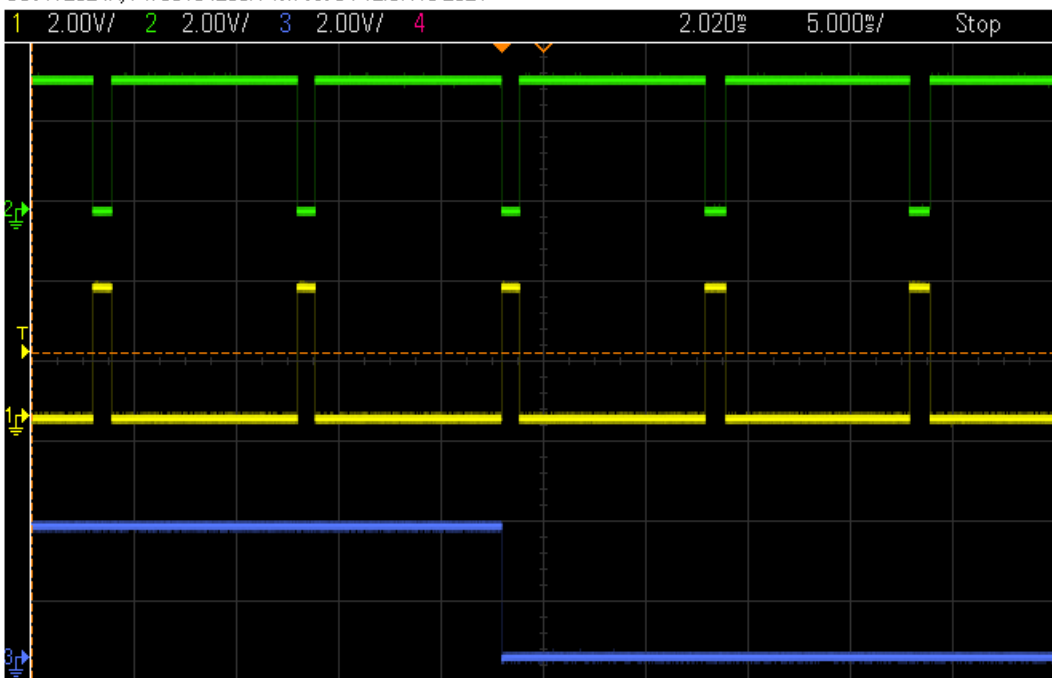
```
00086:
00087: /* Configure the system clock */
00088: SystemClock_Config();
00089:
00090: /* USER CODE BEGIN SysInit */
00091:
00092: /* USER CODE END SysInit */
00093:
00094: /* Initialize all configured peripherals */
00095: MX_GPIO_Init();
00096: MX_USART2_UART_Init();
00097: MX_TIM1_Init();
00098: MX_TIM2_Init();
00099: /* USER CODE BEGIN 2 */
00100: HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
00101: HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
00102: HAL_TIM_Base_Start_IT(&htim2);
00103: /* USER CODE END 2 */
00104:
00105: /* Infinite loop */
00106: /* USER CODE BEGIN WHILE */
.....
```

6

✓ 과제: PWM과 Timer 연동 실습:

- 이렇게 되면, 500[ms](Timer 2) 마다 Timer 1의 PWM Ch1과 Ch2 Duty Cycle이 1%~99% 범위에서 계속 하여 반복적으로 바뀌는 것 오실로스코프로 확인!! (조교 확인 필수)

DSO-X 2024A, MY58104289: Mon Oct 04 12:37:18 2021



Help Menu

Getting Started

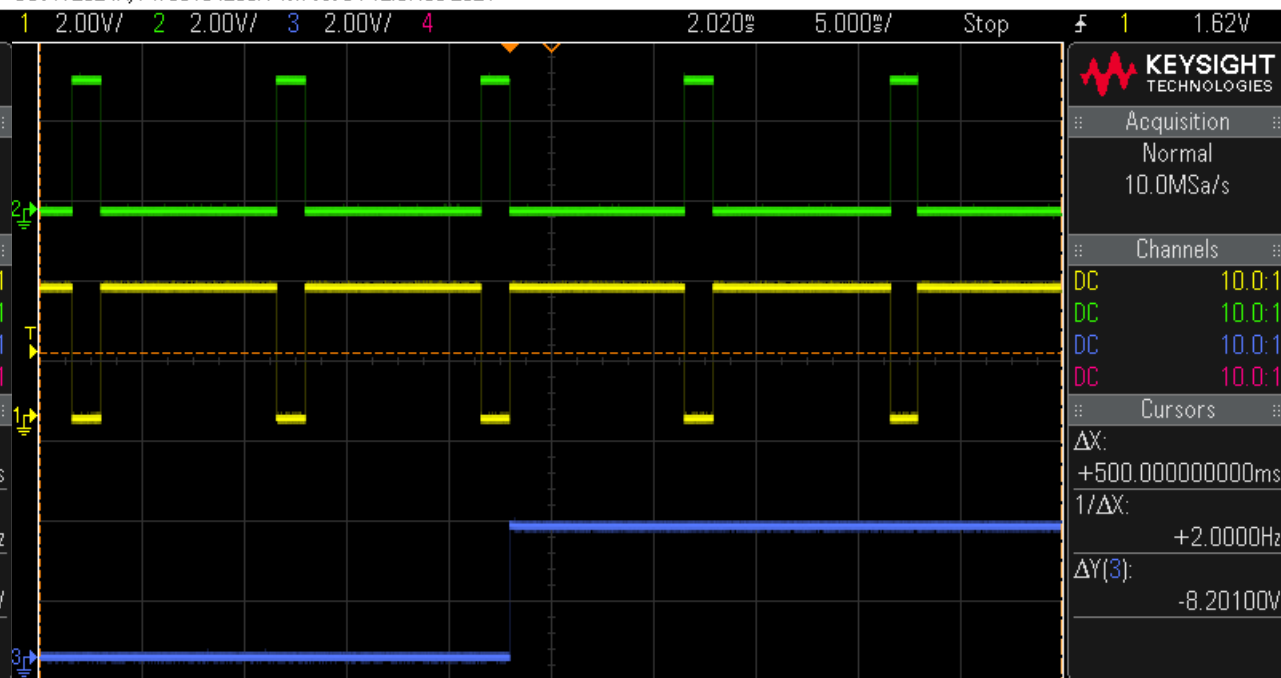
About Oscilloscope

Language English

Training Signals

Learn About 30-day Trial

DSO-X 2024A, MY58104289: Mon Oct 04 12:37:30 2021



Help Menu

Getting Started

About Oscilloscope

Language English

Training Signals

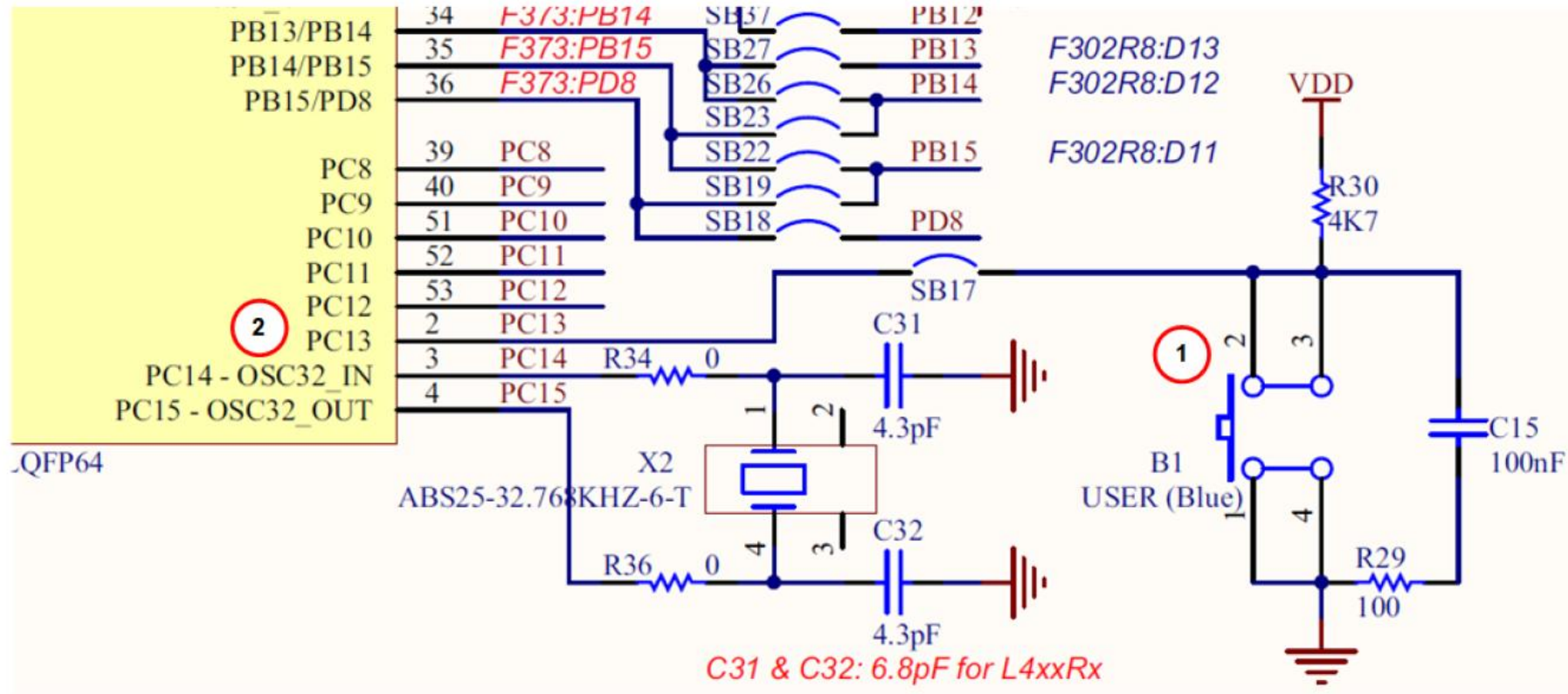
Learn About 30-day Trial

✓ Advanced Control Timer와 General Purpose Timer의 동기화

- 적어도 4개의 동기화된 PWM 신호들을 동시에 생성 가능.
- STM32 MCU의 경우 다음과 같은 3가지 방법의 동기화 기능 제공.
 - 1) Reset Mode :
 - 지정한 trigger 입력이 발생하면 counter 값과 prescaler가 초기화되고, 다시 counting을 시작한다.
 - 2) Gated Mode :
 - counter는 선택한 입력의 level 구간에 대해서만 counting을 수행한다.
 - 예를 들면, Trigger 입력 1이 low level 값을 가지는 동안에만 counting을 수행한다.
 - 3) Triggered Mode :
 - counter는 선택한 입력에 대한 event가 발생하면 counting을 시작한다.
 - 전력 제어를 포함한 다양한 분야에서 사용.

✓ 구현 방법 :

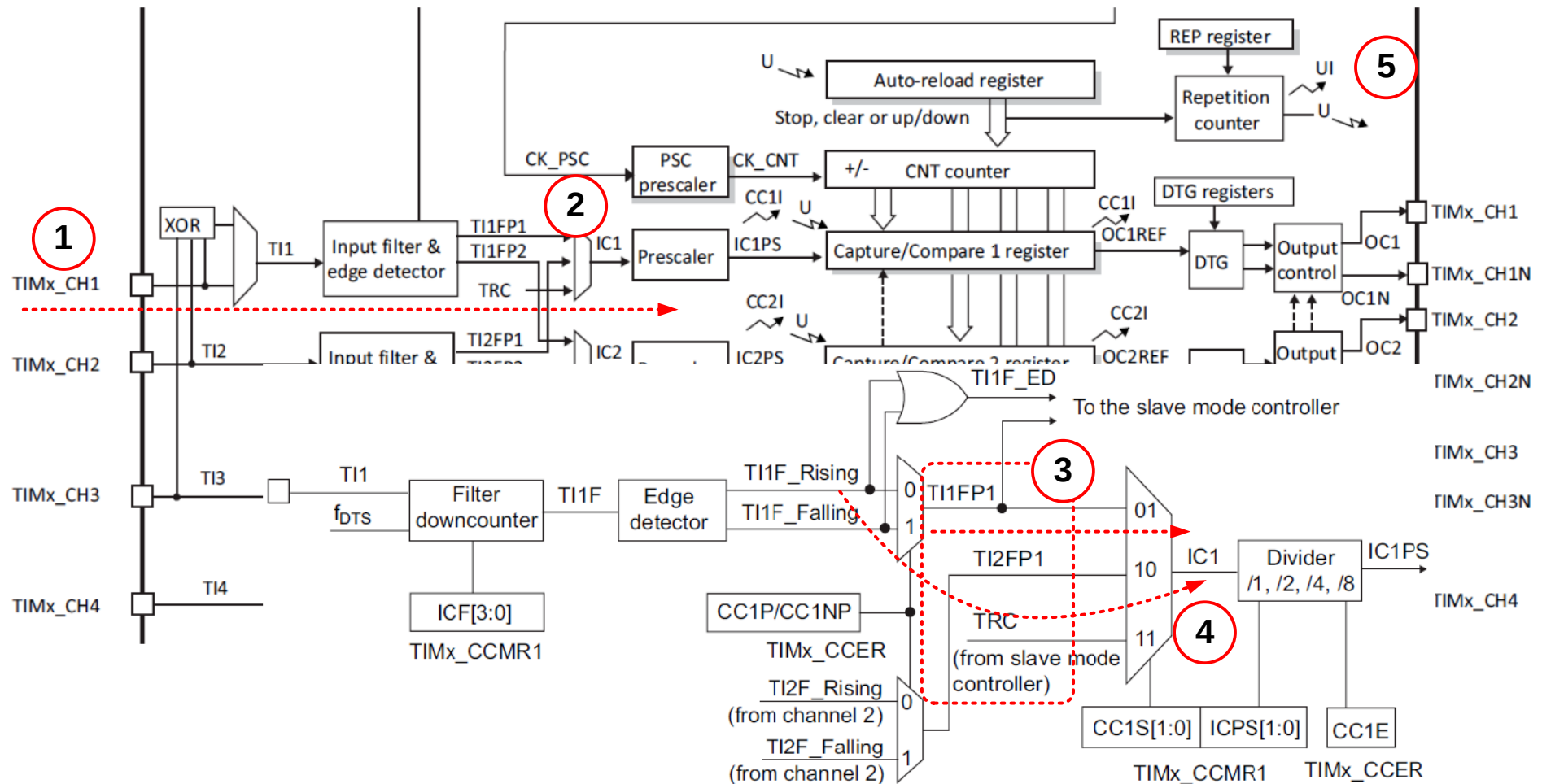
- 파란색 버튼을 Reset Mode 실험을 위한 입력 trigger 전압으로 사용.
- PC13 입력에 대해서 B1 click : 0[V], B1 no-click : 3.3[V]



Reset Mode를 이용한 동기화 - 2

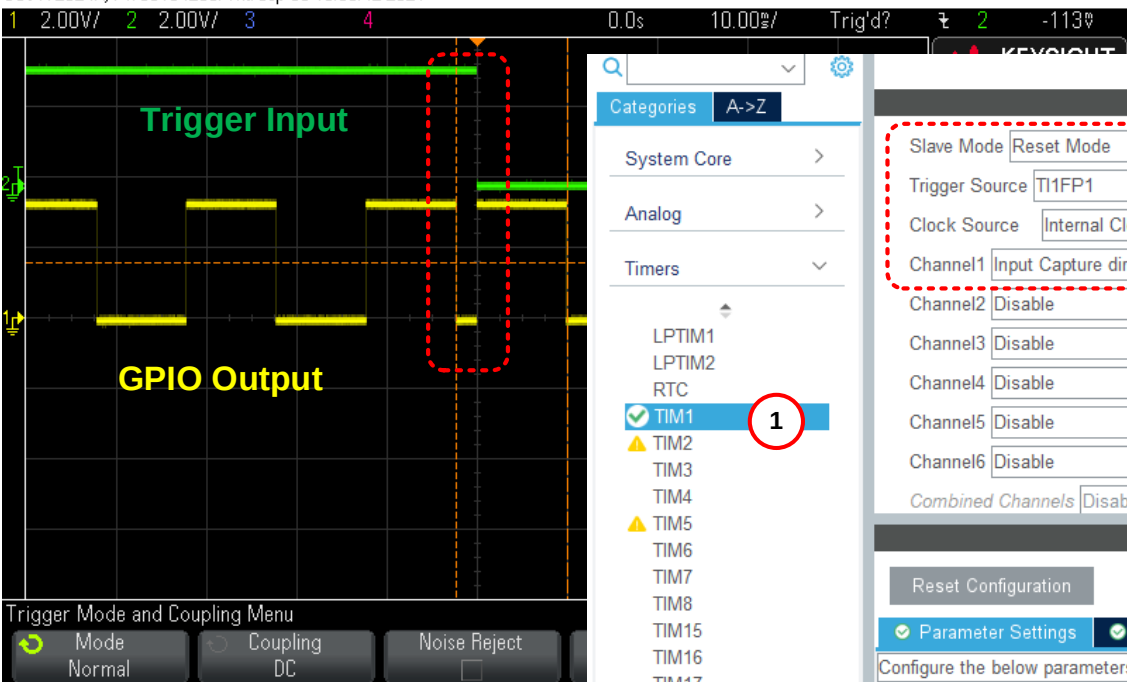
✓ 구현과 실험 방법 :

- Trigger 입력 전압은 ①번 TIMx_CH1으로 주어지고, ⑤번과 같이 UI(update Interrupt) 발생.



Reset Mode를 이용한 동기화 - 3

DSO-X 2024A, MY58104289: Thu Sep 30 10:53:42 2021



TIM1 Mode and Configuration

Categories: A->Z

- System Core >
- Analog >
- Timers >
 - LPTIM1
 - LPTIM2
 - RTC
 - TIM1** (1)
 - TIM2
 - TIM3
 - TIM4
 - TIM5
 - TIM6
 - TIM7
 - TIM8
 - TIM15
 - TIM16
 - TIM17
- Connectivity >
- Multimedia >
- Security >
- Computing >
- Middleware >

Mode

Slave Mode: Reset Mode (2)

Trigger Source: TIM1FP1

Clock Source: Internal Clock

Channel1: Input Capture direct mode

Channel2: Disable

Channel3: Disable

Channel4: Disable

Channel5: Disable

Channel6: Disable

Combined Channels: Disable

Configuration

Reset Configuration

Parameter Settings (3)

Configure the below parameters:

Search (Ctrl+F)

Counter Settings

- Prescaler (PSC - 16 bits value): 80-1
- Counter Mode: Up
- Counter Period (AutoReload Register - 16 bits value): 10000-1
- Internal Clock Division (CKD): No Division
- Repetition Counter (RCR - 8 bits value): 0
- auto-reload preload: Disable
- Slave Mode Controller: Reset Mode

Trigger Output (TRGO) Parameters

Input Capture Channel 1

- Polarity Selection: Falling Edge
- IC Selection: Direct (5)
- Prescaler Division Ratio: No division
- Input Filter (4 bits value): 0

Timer Clock Helper

APB1/2 Timer Clock([MHz]): 80

PSC Register Value("1" 제외): 80

ARR Register Value("1" 제외): 10000

Period([ms]): 10.00 (4)

- ✓ 사용할 Timer :
 - ①번과 같이 TIM1 선택,
- ✓ Slave Mode :
 - ②번과 같이 동기화 방법으로는 Reset Mode 선택,
- ✓ Trigger Source :
 - TI1FP1을 선택 → PA8번 pin 할당.
 - ⑤번의 Polarity Selection에서 보여준 것과 같이 Falling Edge 선택
- ✓ TIMx_CH1을 IC1으로 할당하기 위해서 ②번과 같이 Channel 1에 Input Capture direct mode를 선택
- ✓ Timer 주기는 10[ms] :
 - ③번과 ④번은 시간 간격이 10[ms]로 설정된 것을 보여주고 있다.
- ✓ Reset Mode 동작 확인을 위해서
 - PC13(파란색 버튼)은 TIM1_CH1인 PA8에 연결. PB10은 GPIO port로 설정.

Reset Mode를 이용한 동기화 - 5

TIM1 Mode and Configuration

Mode

- Slave Mode: Reset Mode
- Trigger Source: TI1FP1
- Clock Source: Internal Clock
- Channel1: Input Capture direct mode
- Channel2: Disable
- Channel3: Disable
- Channel4: Disable
- Channel5: Disable
- Channel6: Disable
- Combined Channels: Disable

Configuration

Reset Configuration

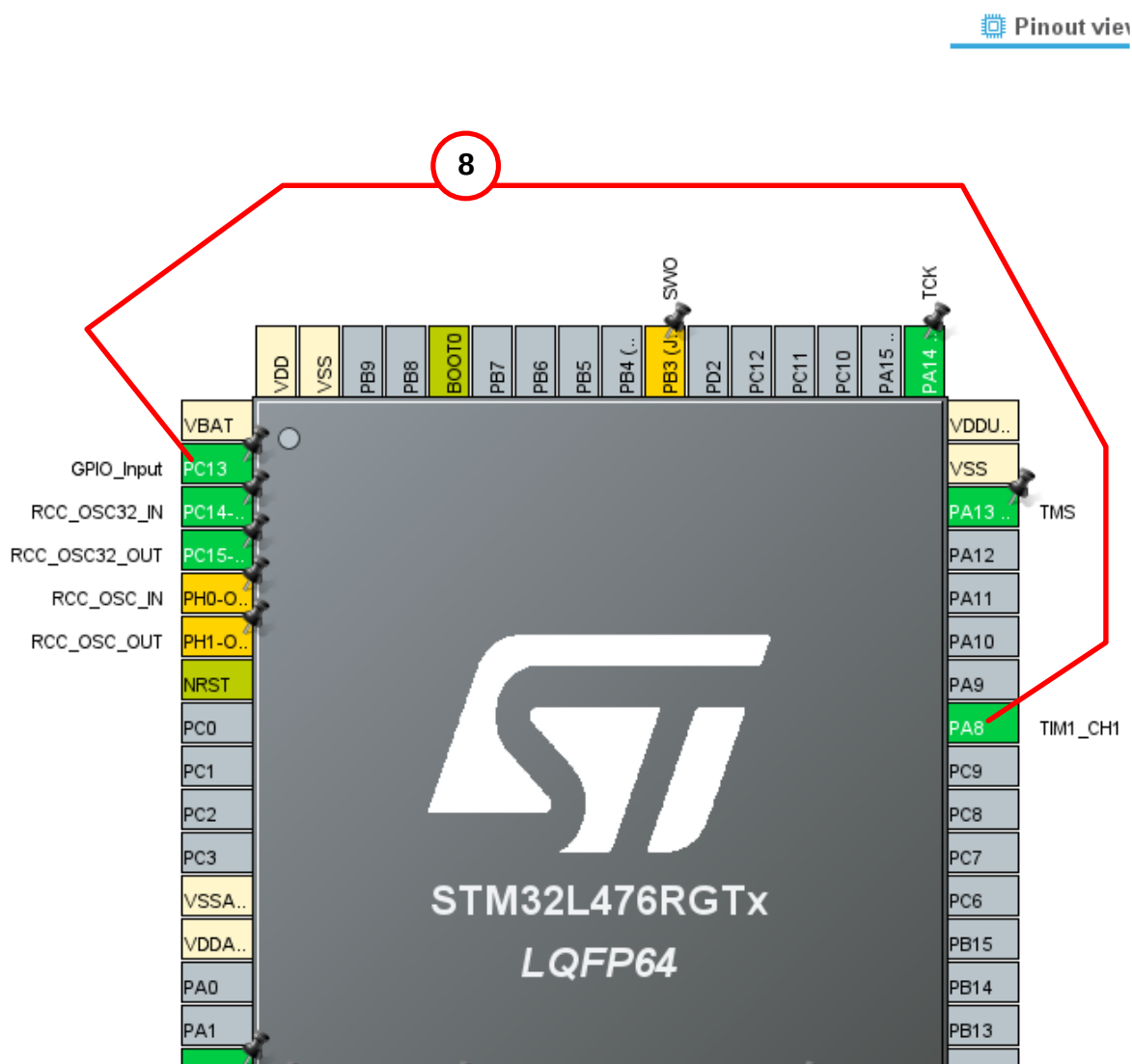
GPIO Settings

- NVIC Settings
- DMA Settings
- Parameter Settings
- User Constants

Search Signals

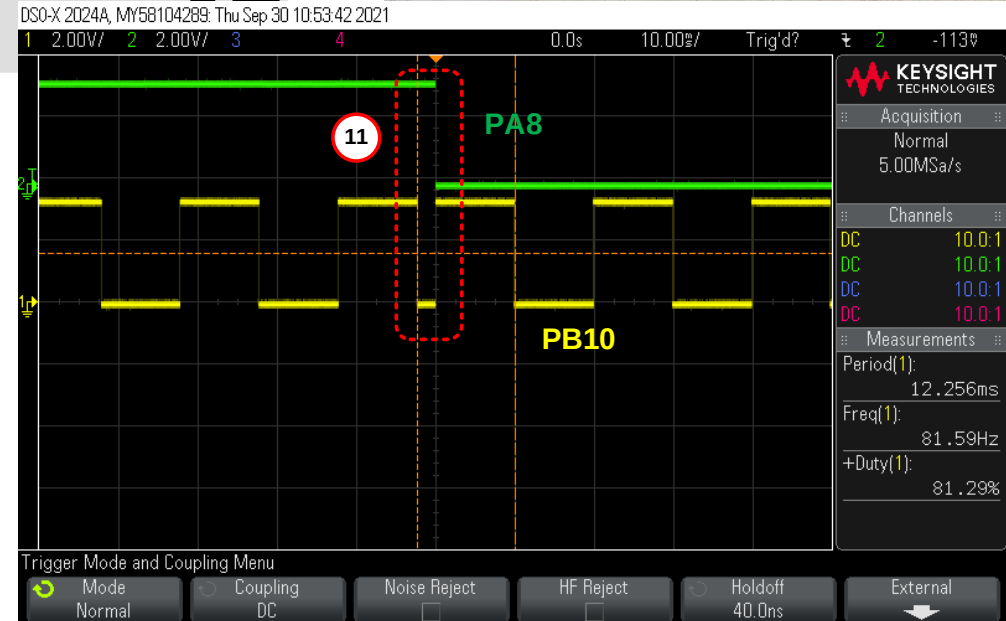
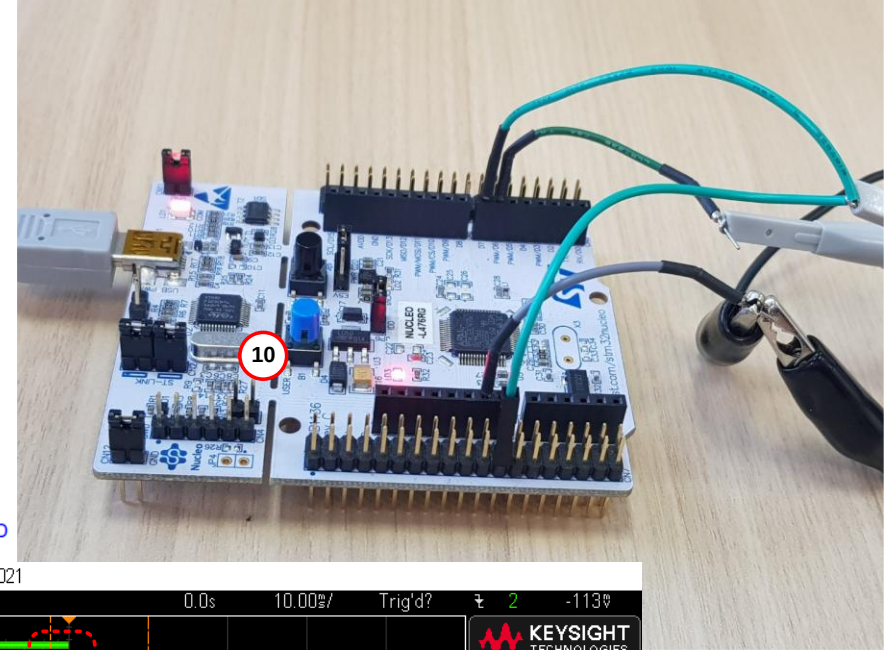
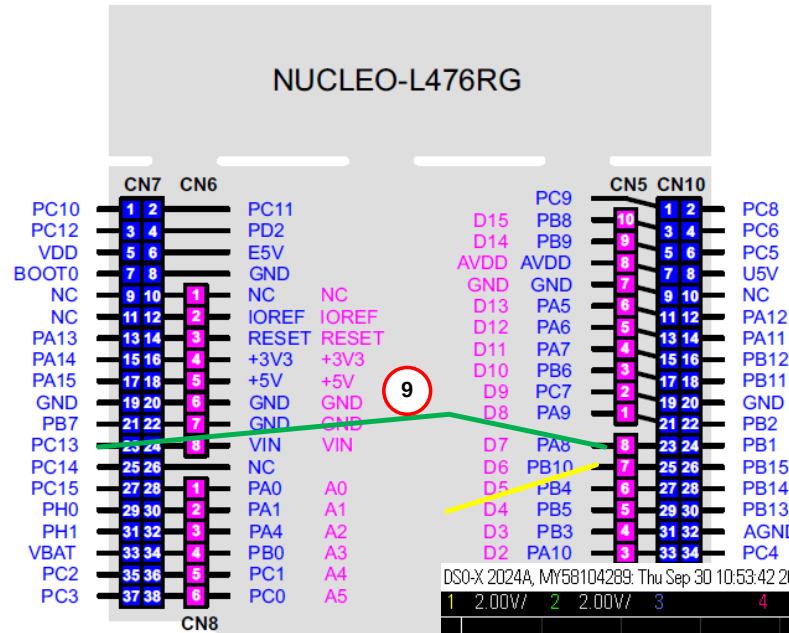
Search (Ctrl+F)

Pin Na...	Signal on ...	GPIO outp...	GPIO mod...
PA8	TIM1_CH1	n/a	Alternate ...



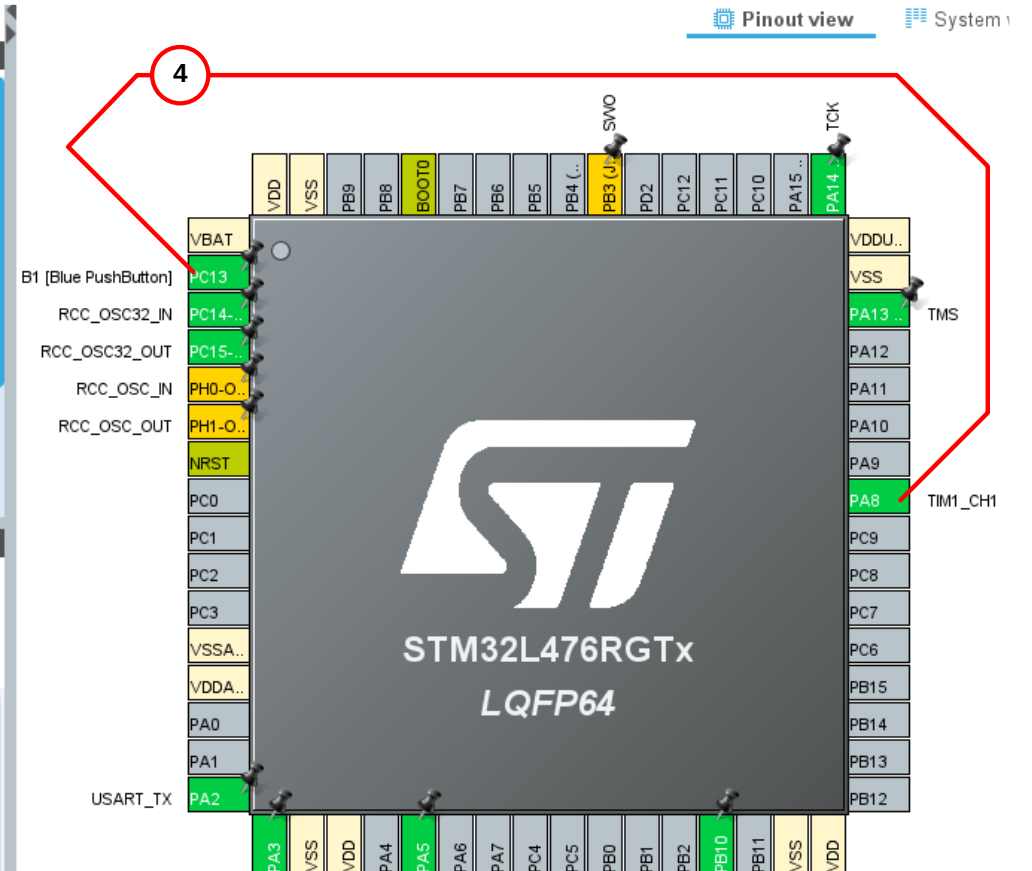
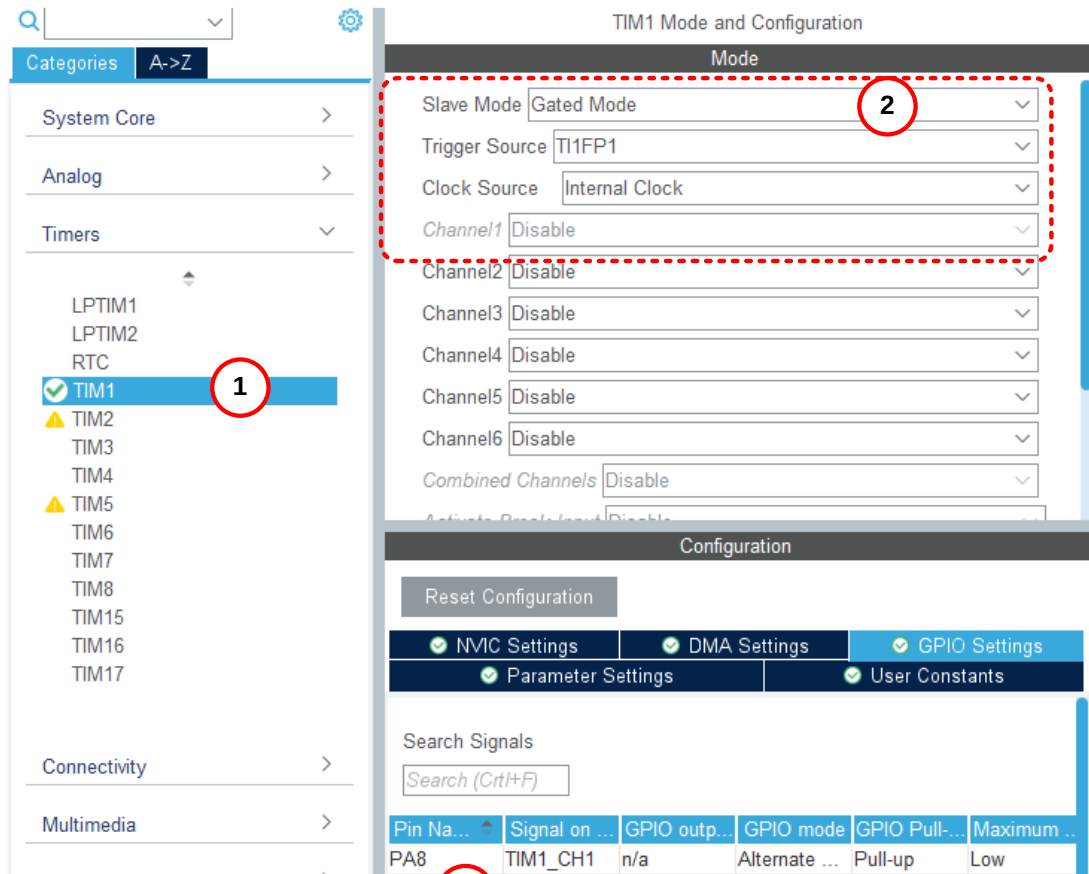
Reset Mode를 이용한 동기화과제

- ✓ PB10번 pin이 10[ms] 마다 toggle되고 있다.
- ✓ 이제, ⑩번에서 보여준 파란색 버튼을 click 할 때마다 Reset Mode가 실행된다.
- ✓ 그리고, ARR, CNT가 초기화가 되어 다시 시작하는 ⑪번 모습을 확인할 수 있다.
- ✓ 즉, 앞서 언급한 것과 같이 지정된 trigger 입력이 발생하면 counter 값과 prescaler가 초기화되고, 다시 counting을 시작하는것 오실로스코프로 확인.



✓ 구현과 실험 방법 :

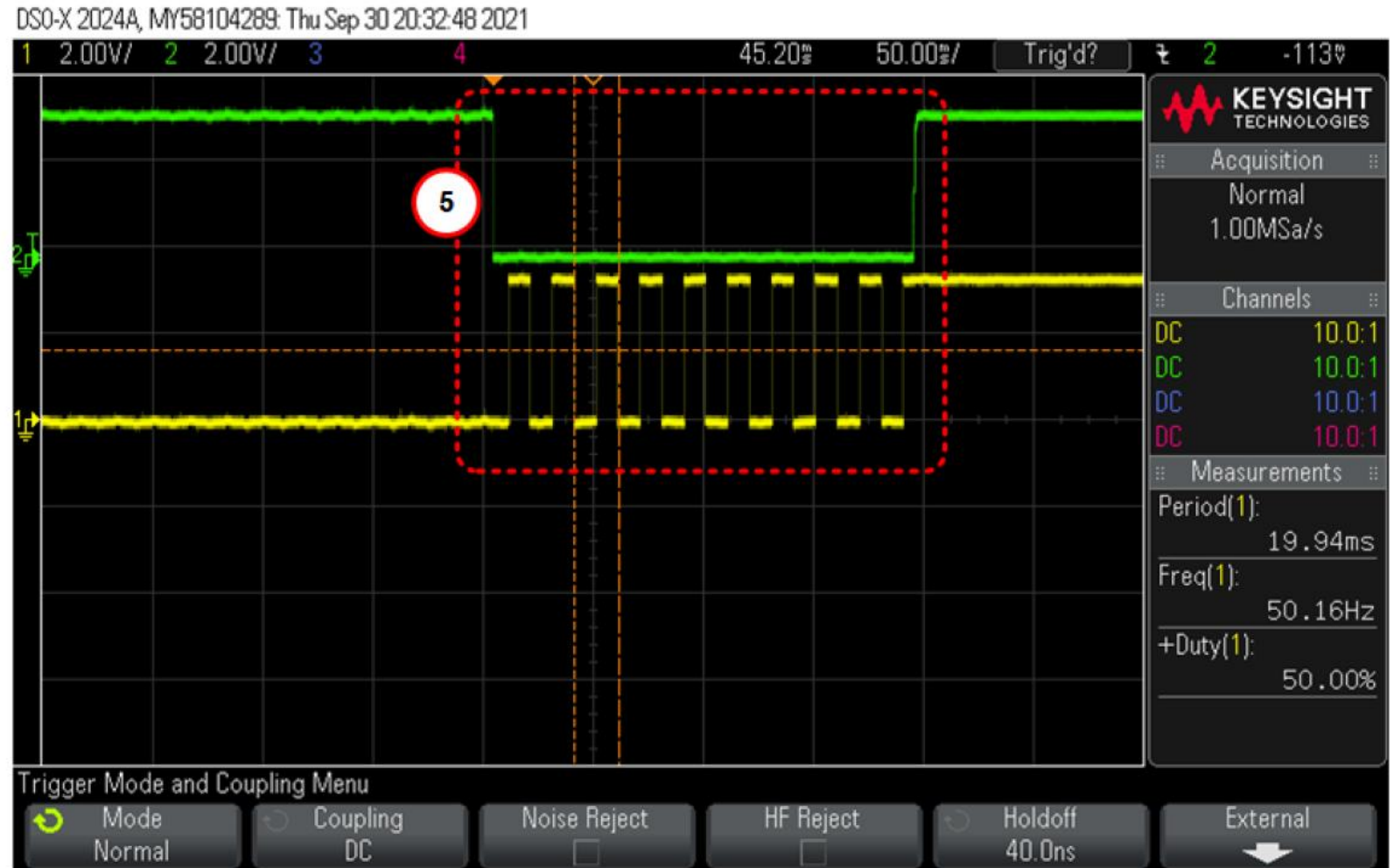
- 실험에 사용할 보드 구성은 Reset Mode와 동일.



3

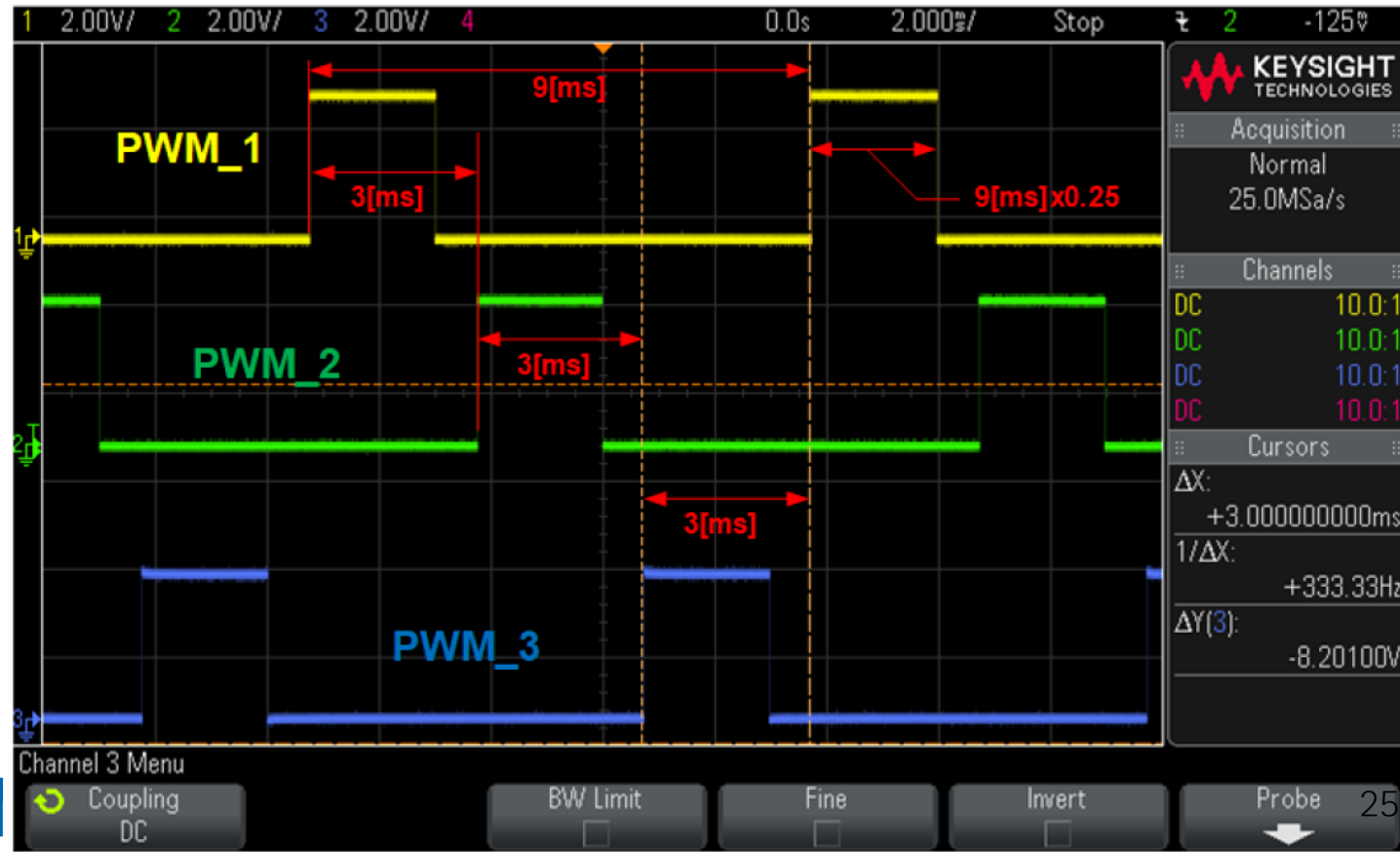
- ✓ 사용할 Timer :
 - ①번과 같이 TIM1 선택,
- ✓ Slave Mode :
 - ②번과 같이 동기화 방법으로는 Gated Mode 선택,
- ✓ Trigger Source :
 - TI1FP1을 선택 → PA8번 pin 할당.
- ✓ PC13번 pin을 할당된 PA8번 pin에 jumper wire로 연결하고, 파란색 버튼을 click 하여 주면, click 되어 있는 동안은 low level이 되고, 이 구간 동안만 Timer 1이 동작할 것이다.

- ✓ Gated mode 동기화 결과 오실로스코프로 측정!
(조교 확인 필수)



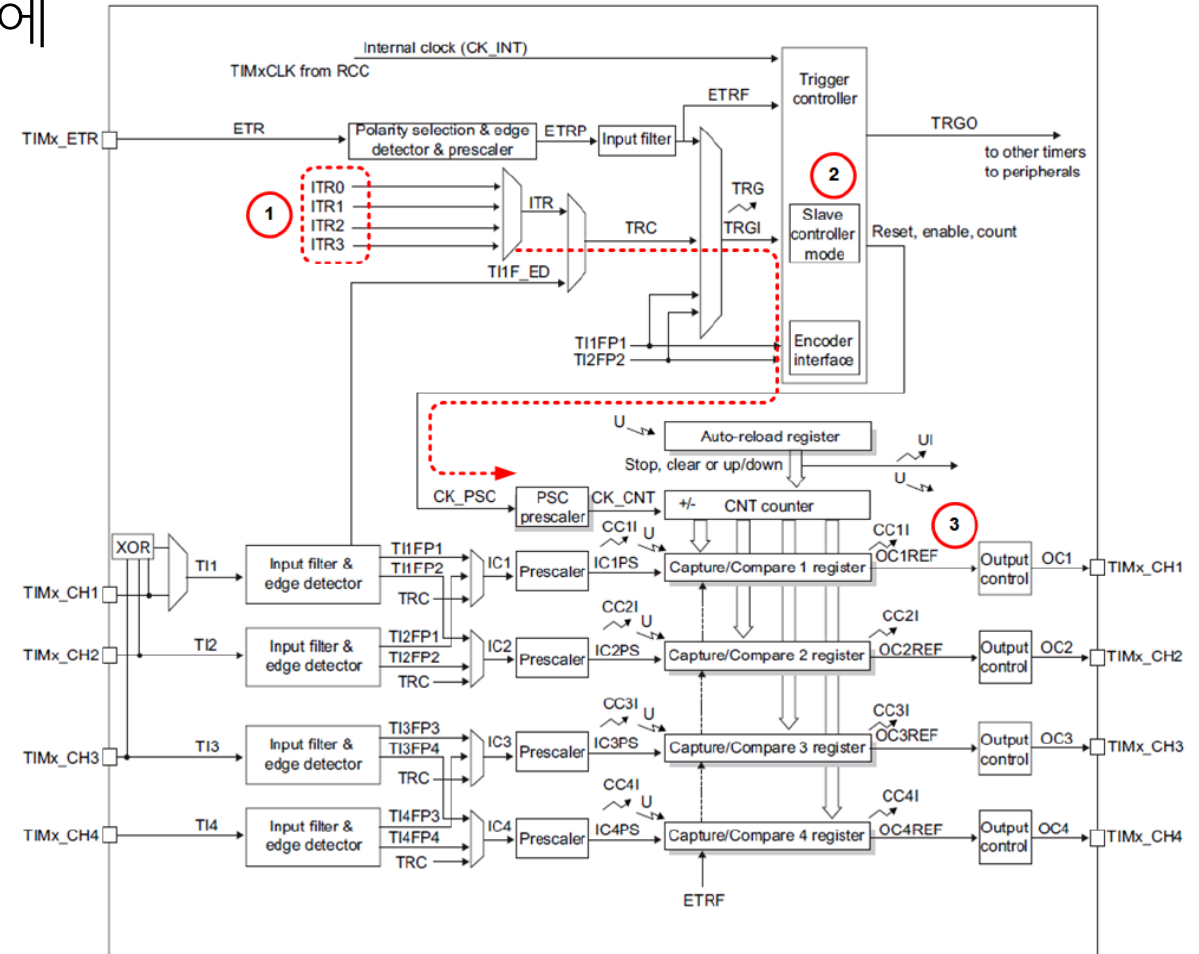
Trigger Mode를 이용한 동기화 - 1

- ✓ Trigger mode를 이용하여 동기화를 수행한다는 것은 생성한 PWM 신호들 사이에 일정한 위상각을 유지하도록 만드는 것이다.
- ✓ PWM_1, PWM_2, 그리고, PWM_3은 모두 동일한 9[ms] 주기와 25% duty cycle을 갖고 있다.
- ✓ 3개의 PWM 신호들은 서로 120도 위상차를 유지하고 있다.
- ✓ 360도의 120도 위상차는 $120/360=1/3$ 인 것과 같이 전체 360도에 해당하는 9[ms]에서 3[ms]만 high level이므로 $3/9=1/3$ 즉, 3[ms]는 120도에 해당한다.



Trigger Mode를 이용한 동기화 - 2

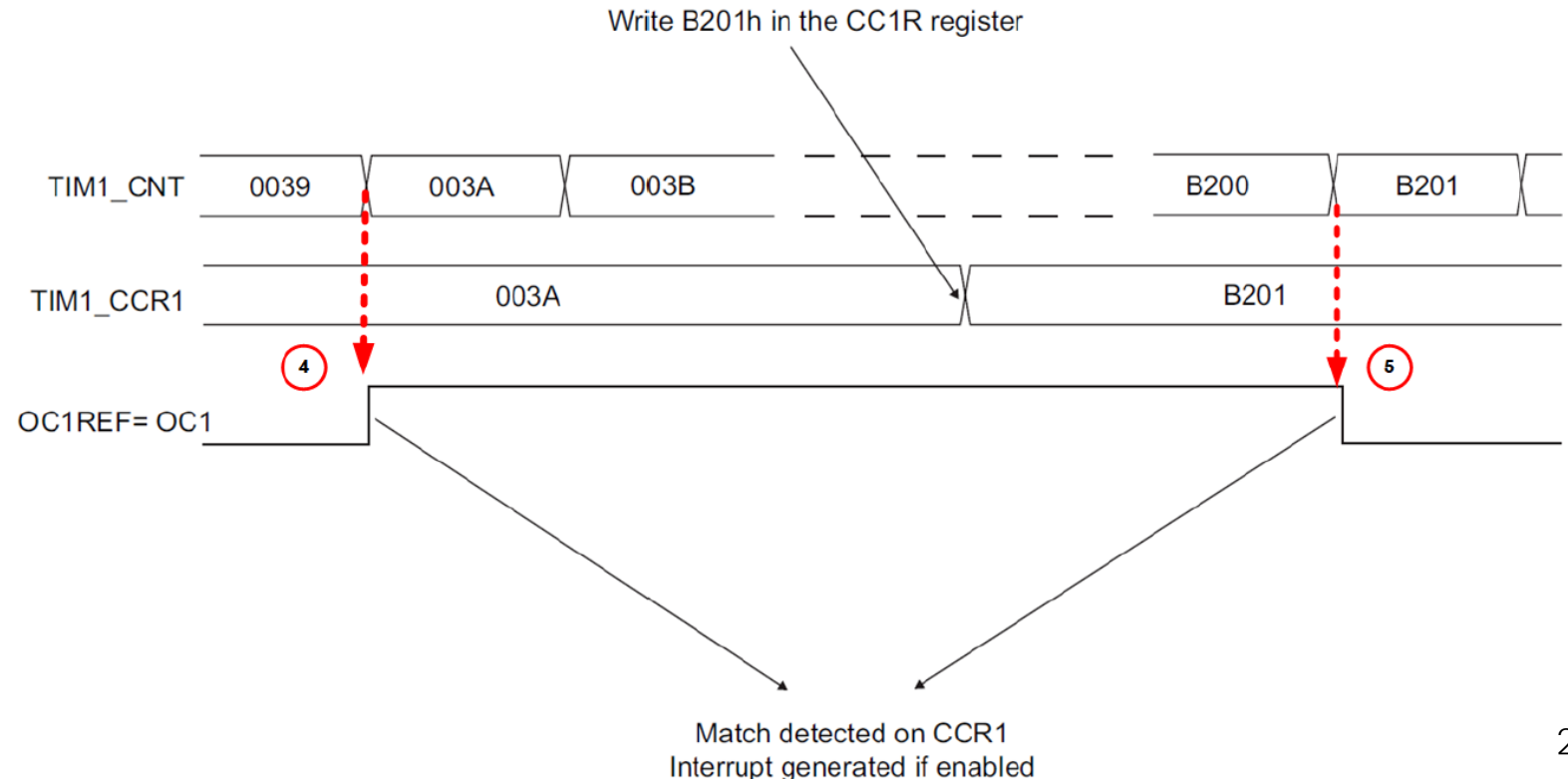
- ✓ ①번을 보면, 내부 trigger 신호로 ITR0~ITR3을 사용할 수 있고, 이들은 TRGI 즉, trigger interrupt 신호 발생에 대한 source인 것을 알 수 있다.
- ✓ 또한, ②번에서 보여준 것과 같이 Slave controller mode에서 사용해야 하며, 결국, CNT counting을 수행하기 위한 clock 즉, CK_PSC를 생성할 때, 점선으로 표시한 것과 같이 동기화를 수행하는 역할을 수행한다.
- ✓ 구체적으로 ③번과 같이 OCxREF에 설정한 값과 일치할 때 interrupt를 발생하여 이것을 내부 trigger ITR0~ITR3의 source로 사용한다.



[그림 7.2.3-2] 내부 trigger 신호 사용 방법.

Trigger Mode를 이용한 동기화 - 3

- ✓ Timer1의 CNT counter 값이 지정한 방향 예를 들면, Up counting을 하다가 ④번처럼 CCR1에 설정한 값과 일치하는 순간에 interrupt가 발생한다.
- ✓ 그리고, CNT counter가 계속해서 counting을 하는 동안 CCR1의 값을 0xB201로 설정 값을 바꾸어 준다. 그러면, 계속해서 Up counting하다가 역시, ⑤번처럼 CCR1에 설정한 새로운 값과 같으면, interrupt를 발생시키는데, 이들 interrupt는 내부 trigger ITR0~ITR3의 source로 사용된다는 의미
- ✓ 다른 PWM 신호들을 생성할 때에 trigger 신호로 사용될 수 있다는 의미이며, 이들 생성된 PWM 신호는 결국, trigger 신호에 동기 되어 만들어진다.



- ✓ TIM1의 Channel1은 PWM 신호를 출력하기 위해서 사용하고, Channel2의 CCR2에 설정한 값이 CNT counting 값과 일치하게 되면, interrupt가 발생하도록 하고, 그 발생한 interrupt를 TIM2에 대한 동기화 신호로 사용하고 싶다면, TIM2는 slave mode로 동작해야 할 것이다.
- ✓ TIM2는 GP timer 이므로 [표 7.2.3-1]에서 보여준 것과 같이 내부 trigger ITR0에 의해서 TIM1과 연결된다.

Slave TIM	ITR0 (TS = 000)	ITR1 (TS = 001)	ITR2 (TS = 010)	ITR3 (TS = 011)
TIM2	TIM1	TIM8/OTG_FS_SOF ⁽¹⁾	TIM3	TIM4
TIM3	TIM1	TIM2	TIM15	TIM4
TIM4	TIM1	TIM2	TIM3	TIM8
TIM5	TIM2	TIM3	TIM4	TIM8

1. Depends on the bit ITR1_RMP in TIM2_OR1 register.

[표 7.2.3-1] GP Timer 내부 trigger 연결도.

- ✓ TIM1의 CCR2에 설정한 값에 의해서 발생한 OC2REF 신호의 edge는 interrupt를 생성시키고, 이 interrupt는 내부 trigger 신호 ITR0에 의해서 TIM2에 전달되어 동기화를 이루게 되는 것이다.

- TIM1에 대한 CubeMX 설정 화면 :

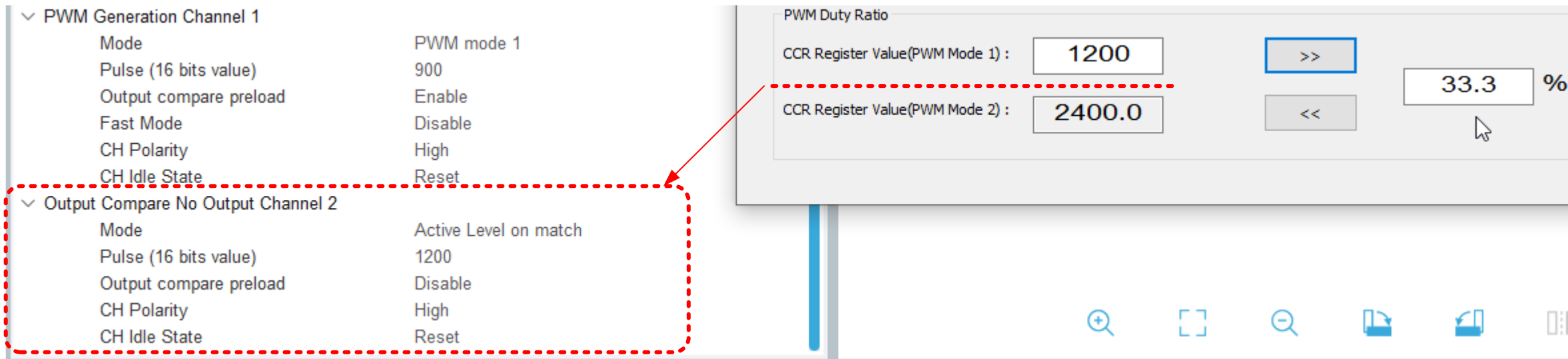
The screenshot displays the STM32CubeMX configuration environment for the TIM1 timer. The left-hand 'Categories' pane lists various system components, with 'Timers' expanded and TIM1 selected. The central workspace is divided into 'Mode' and 'Configuration' sections. The 'Mode' section shows settings for Slave Mode (Disable), Trigger Source (Disable), Clock Source (Internal Clock), and three channels (Channel1: PWM Generation CH1, Channel2: Output Compare No Output, Channel3: Disable). The 'Configuration' section includes tabs for NVIC Settings, DMA Settings, GPIO Settings, Parameter Settings, and User Constants. The 'Parameter Settings' tab is active, showing a search bar and a list of parameters for the counter and trigger output. A 'Timer Clock Helper' dialog box is overlaid on the right, providing a summary of the timer's clock configuration: APB1/2 Timer Clock is 80 MHz, PSC Register Value is 200, and ARR Register Value is 3600, resulting in a 9.00 ms period. It also shows the PWM Duty Ratio with CCR Register Value 1 at 900 and CCR Register Value 2 at 2700.0, resulting in a 25.0% duty cycle.

✓ TIM1에 대한 CubeMX 설정 요약 :

- Timer 1은 trigger 신호를 받지 않으므로 ⑤번과 같이 Slave Mode는 disable을 선택.
- Trigger Source도 역시, disable을 선택.
- Channel1 즉, TIM1_CH1 pin으로 PWM1을 출력하도록 설정.
- Channel2 즉, TIM1_CH2의 경우에는 출력은 하지 않고, 비교만하도록 선택해 준다.
- 즉, OC2REF 신호를 이용하여 ITR0를 생성하여 Timer2에 trigger 신호를 출력할 것이다.
- APB2=80[MHz]인 경우에 ARR의 값이 정확히 1/3 등분되는 PSC와 ARR 값을 찾아본다.
- 예를 들면, 1주기가 9[ms]이면서 PSC=200, ARR=3600과 같이 ARR의 값이 1/3로 나누어 떨어지는 값 즉, 3600과 같은 값을 찾는다.
- 결국, 360도의 1/3이 120도이므로 1200의 값을 OC2REF에 해당하는 CCR2에 할당하면 될 것이다.

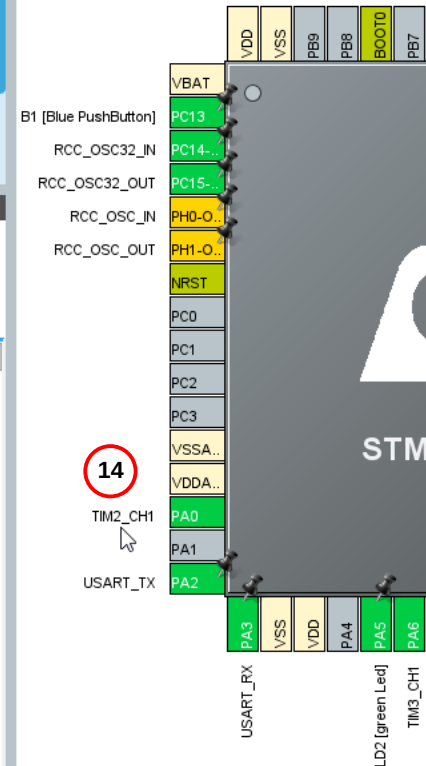
- ⑤번에서 보여준 것과 같이 현재, Channel2에 Output Compare No Output을 선택하였으므로 ⑦번과 같이 Trigger Event Selection TRGO에 나열되는 항목 중에서 OC2REF를 선택해 주어야 내부 trigger 신호인 ITR0에 연결되는 slave timer 즉, [표 7.2.3-1]나와 있듯이 Timer2 와 동기를 맞출 수 있다.
- 일단, Timer 1의 Channel1로 출력할 PWM1의 duty cycle을 25%로 설정하기 위해서 필요한 CubeMX의 PWM Generation Channel 1에 속하는 파라미터들 중에서 CCR1의 값을 결정해야 한다.

- 이어서 ⑨번 즉, Channel2의 OC2REF에 1200을 할당하는 부분은 다음을 참조하면 된다.



- Mode로는 CNT counting 값이 1200일 때, 내부 trigger 신호 즉, ITR0가 active 되도록 Active Level on match로 설정하였다. 1200을 설정하였을 때에 전체 100%의 1/3인 33.3%인 것을 알 수 있으며, 앞서 계산 것과 같이 전체 3600의 1/3에 해당하는 trigger 신호이다.

- 이제, Timer1에 대한 설정이 끝났으면, 다음과 같이 Timer2에 대한 설정을 수행한다.
- Timer 1 설정과 비슷한 개념이다.
- 단지, Timer 1이 제공하는 내부 trigger 신호 ITR0를 사용해야 하므로 ⑪번과 같이 Slave Mode로는 Trigger Mode를 선택하고, Trigger Source로는 ITR0를 선택하였다.
- 그리고, Timer3과 동기를 맞추기 위해서 즉, trigger 신호를 제공하기 위해서 Channel 2에서 OC2REF를 사용하고 있다.
- 주의 할 것은 여기서의 OC2REF는 앞서 timer1에 속하는 OC2REF가 아닌 timer 2에 속하는 OC2REF이다.



- 그리고, 다음과 같이 Timer3에 대한 설정을 수행한다.
- Timer 3가 Slave Mode인 경우에 [표 7.2.3-1]에서 보여준 것과 같이 ITR1 내부 trigger 신호에 의해서 Timer 2가 연결된다는 것이다.
- 그러므로, ⑩번과 같이 Trigger Source에 ITR1을 할당한다.

STM32CubeMX TIM3 Mode and Configuration

Mode

- Slave Mode: Trigger Mode (16)
- Trigger Source: ITR1
- Clock Source: Internal Clock
- Channel1: PWM Generation CH1
- Channel2: Disable
- Channel3: Disable
- Channel4: Disable

Configuration

Reset Configuration

☒ NVIC Settings
 ☒ DMA Settings
 ☒ GPIO Settings
 ☒ Parameter Settings
 ☒ User Constants

Configure the below parameters :

Search (Ctrl+F)

▼ Counter Settings

- Prescaler (PSC - 16 bits value): 200-1
- Counter Mode: Up
- Counter Period (AutoReload Register - ...): 3600-1
- Internal Clock Division (CKD): No Division
- auto-reload preload: Disable
- Slave Mode Controller: Trigger Mode

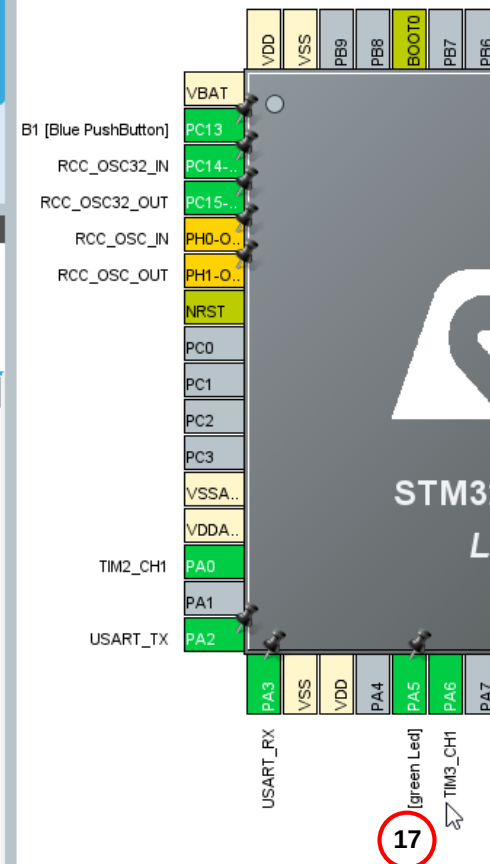
▼ Trigger Output (TRGO) Parameters

- Master/Slave Mode (MSM bit): Disable (Trigger input effect not delayed)
- Trigger Event Selection TRGO: Reset (UG bit from TIMx_EGR)

> Clear Input

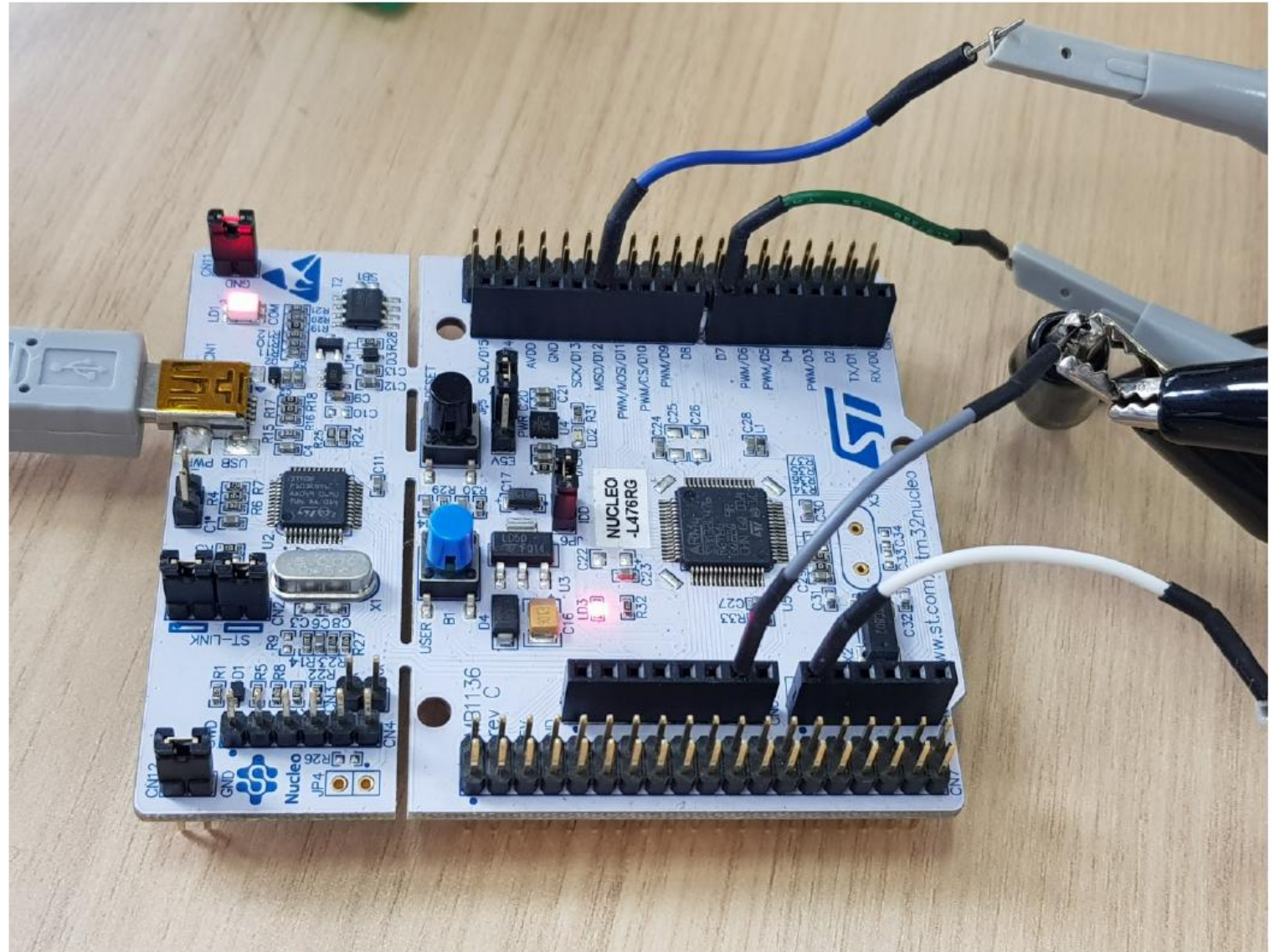
▼ PWM Generation Channel 1

- Mode: PWM mode 1
- Pulse (16 bits value): 900
- Output compare preload: Enable
- Fast Mode: Disable
- CH Polarity: High



Trigger Mode를 이용한 동기화 실습 과제

- ✓ 120도 위상차를 가진, 3개의 PWM 생성된 것 오실로스코프로 측정! (조교 확인 필수)



[그림 7.2.3-8] 120도 위상차 PWM 실험 모습.